
12.2 Merge-Sort

12.2.1 Divide-and-Conquer

The first two algorithms we describe in this chapter, merge-sort and quick-sort, use recursion in an algorithmic design pattern called *divide-and-conquer*. We have already seen the power of recursion in describing algorithms in an elegant manner (see Chapter 4). The divide-and-conquer pattern consists of the following three steps:

1. **Divide:** If the input size is smaller than a certain threshold (say, one or two elements), solve the problem directly using a straightforward method and return the solution so obtained. Otherwise, divide the input data into two or more disjoint subsets.
2. **Conquer:** Recursively solve the subproblems associated with the subsets.
3. **Combine:** Take the solutions to the subproblems and merge them into a solution to the original problem.

Using Divide-and-Conquer for Sorting

We will first describe the merge-sort algorithm at a high level, without focusing on whether the data is an array-based (Python) list or a linked list; we will soon give concrete implementations for each. To sort a sequence S with n elements using the three divide-and-conquer steps, the merge-sort algorithm proceeds as follows:

1. **Divide:** If S has zero or one element, return S immediately; it is already sorted. Otherwise (S has at least two elements), remove all the elements from S and put them into two sequences, S_1 and S_2 , each containing about half of the elements of S ; that is, S_1 contains the first $\lfloor n/2 \rfloor$ elements of S , and S_2 contains the remaining $\lceil n/2 \rceil$ elements.
2. **Conquer:** Recursively sort sequences S_1 and S_2 .
3. **Combine:** Put back the elements into S by merging the sorted sequences S_1 and S_2 into a sorted sequence.

In reference to the divide step, we recall that the notation $\lfloor x \rfloor$ indicates the *floor* of x , that is, the largest integer k , such that $k \leq x$. Similarly, the notation $\lceil x \rceil$ indicates the *ceiling* of x , that is, the smallest integer m , such that $x \leq m$.

We can visualize an execution of the merge-sort algorithm by means of a binary tree T , called the *merge-sort tree*. Each node of T represents a recursive invocation (or call) of the merge-sort algorithm. We associate with each node v of T the sequence S that is processed by the invocation associated with v . The children of node v are associated with the recursive calls that process the subsequences S_1 and S_2 of S . The external nodes of T are associated with individual elements of S , corresponding to instances of the algorithm that make no recursive calls.

Figure 12.1 summarizes an execution of the merge-sort algorithm by showing the input and output sequences processed at each node of the merge-sort tree. The step-by-step evolution of the merge-sort tree is shown in Figures 12.2 through 12.4.

This algorithm visualization in terms of the merge-sort tree helps us analyze the running time of the merge-sort algorithm. In particular, since the size of the input sequence roughly halves at each recursive call of merge-sort, the height of the merge-sort tree is about $\log n$ (recall that the base of $\log$ is 2 if omitted).

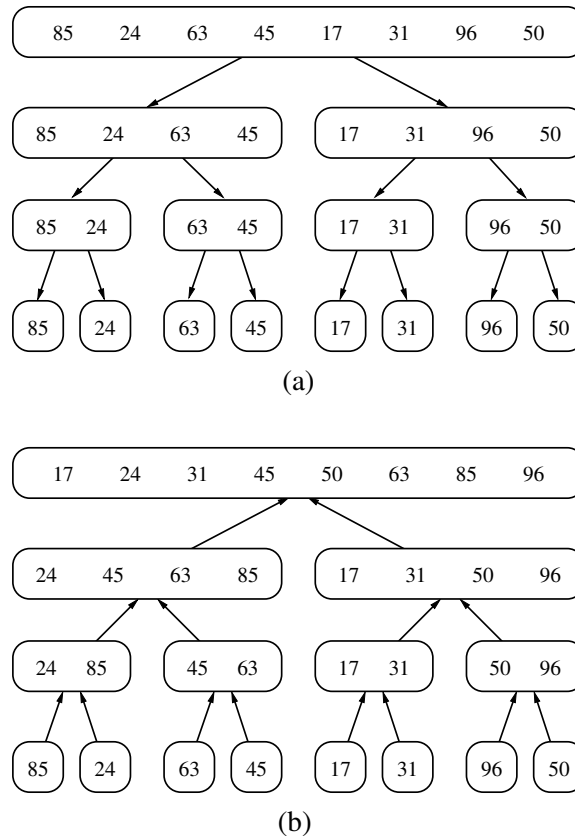


Figure 12.1: Merge-sort tree T for an execution of the merge-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T ; (b) output sequences generated at each node of T .

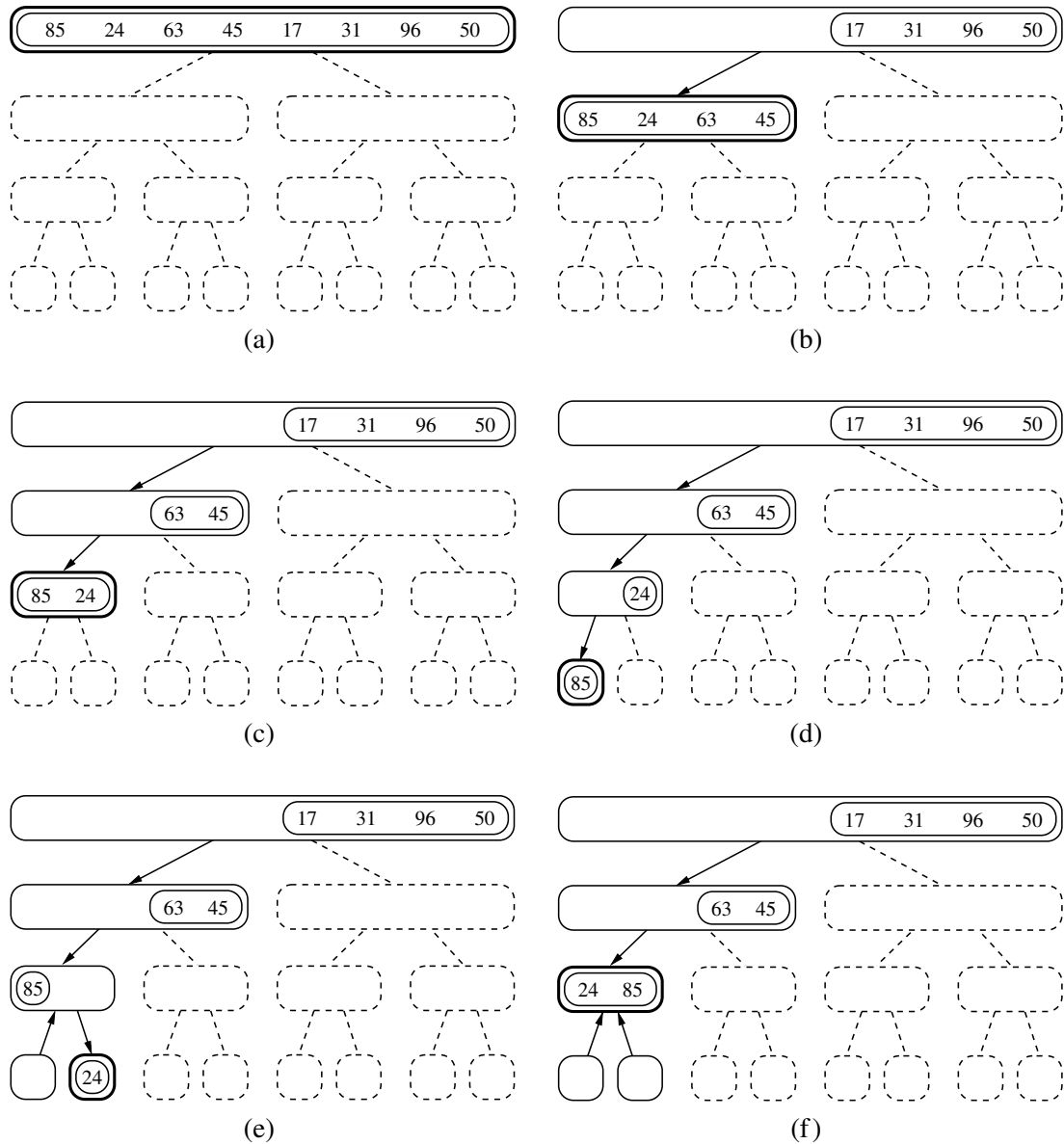


Figure 12.2: Visualization of an execution of merge-sort. Each node of the tree represents a recursive call of merge-sort. The nodes drawn with dashed lines represent calls that have not been made yet. The node drawn with thick lines represents the current call. The empty nodes drawn with thin lines represent completed calls. The remaining nodes (drawn with thin lines and not empty) represent calls that are waiting for a child invocation to return. (Continues in Figure 12.3.)

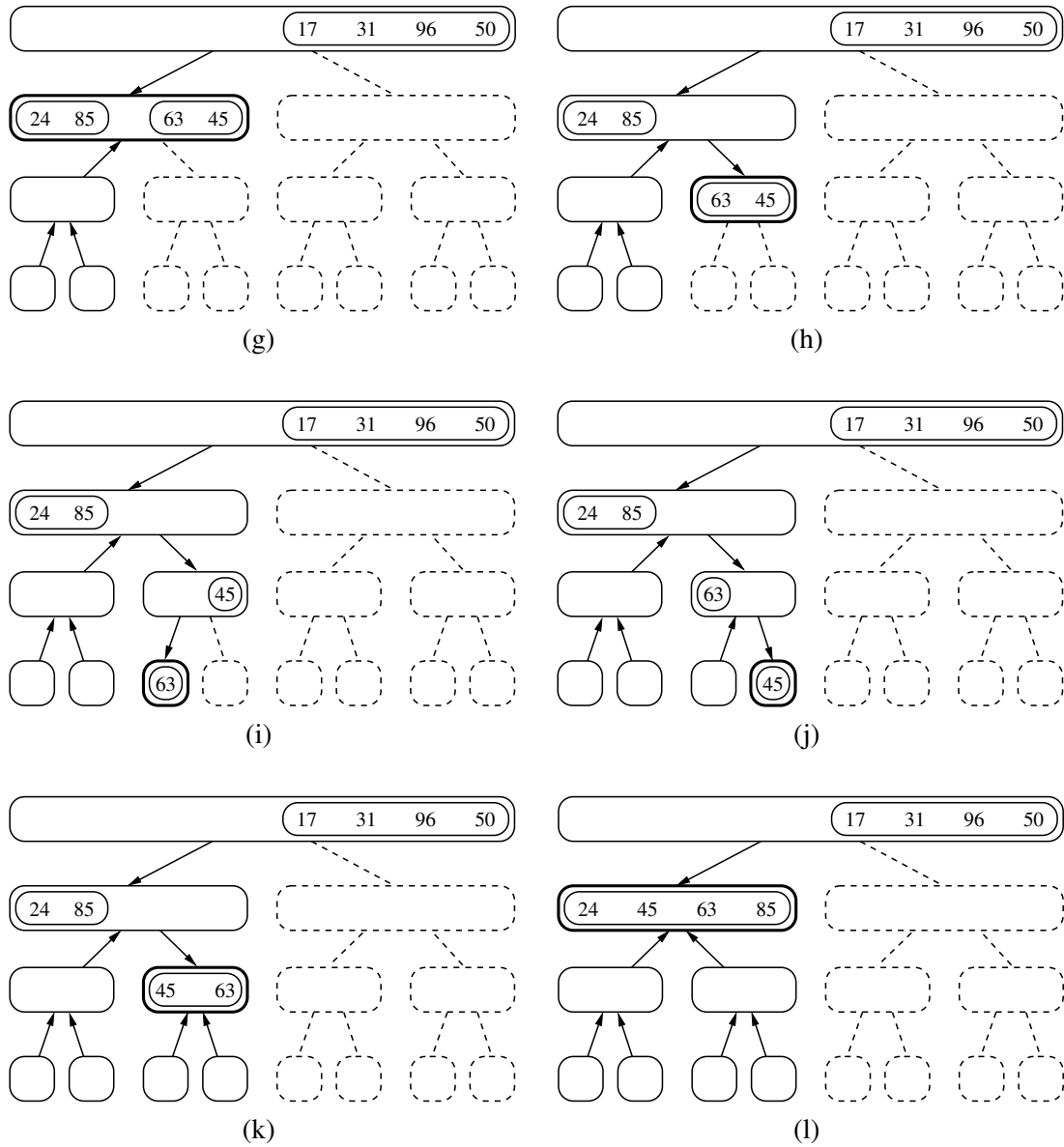


Figure 12.3: Visualization of an execution of merge-sort. (Combined with Figures 12.2 and 12.4.)

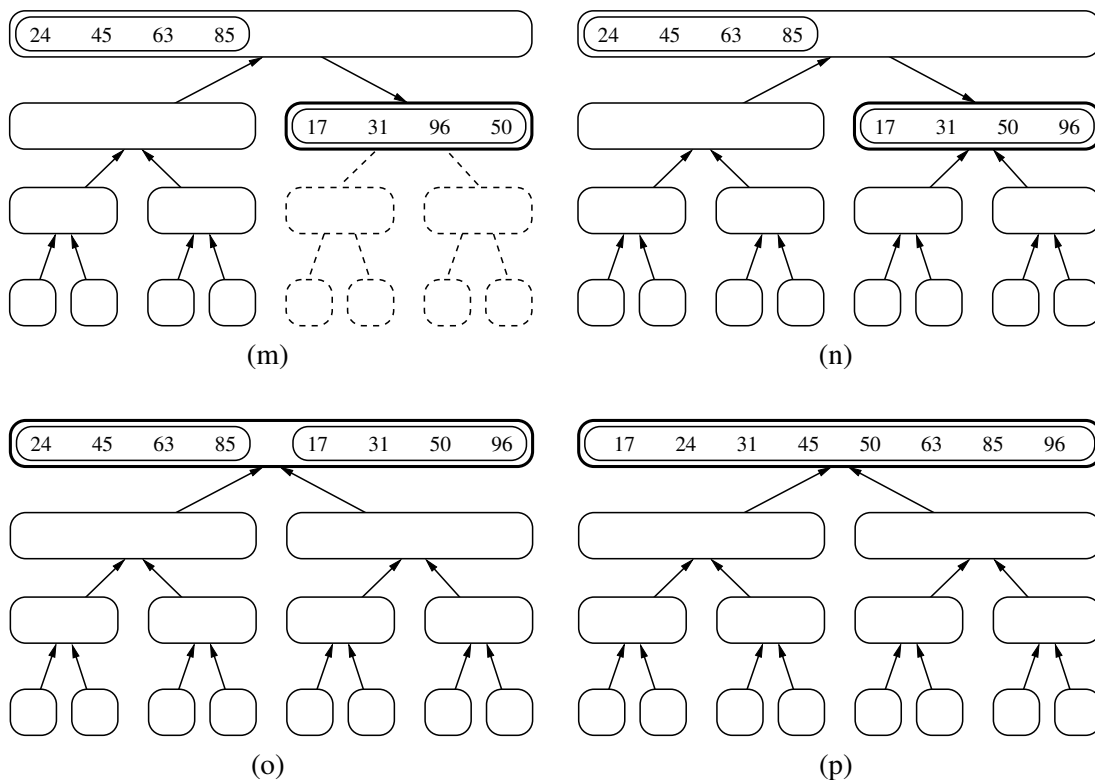


Figure 12.4: Visualization of an execution of merge-sort (continued from Figure 12.3). Several invocations are omitted between (m) and (n). Note the merging of two halves performed in step (p).

Proposition 12.1: *The merge-sort tree associated with an execution of merge-sort on a sequence of size n has height $\lceil \log n \rceil$.*

We leave the justification of Proposition 12.1 as a simple exercise (R-12.1). We will use this proposition to analyze the running time of the merge-sort algorithm.

Having given an overview of merge-sort and an illustration of how it works, let us consider each of the steps of this divide-and-conquer algorithm in more detail. Dividing a sequence of size n involves separating it at the element with index $\lceil n/2 \rceil$, and recursive calls can be started by passing these smaller sequences as parameters. The difficult step is combining the two sorted sequences into a single sorted sequence. Thus, before we present our analysis of merge-sort, we need to say more about how this is done.

12.2.2 Array-Based Implementation of Merge-Sort

We begin by focusing on the case when a sequence of items is represented as an (array-based) Python list. The merge function (Code Fragment 12.1) is responsible for the subtask of merging two previously sorted sequences, S_1 and S_2 , with the output copied into S . We copy one element during each pass of the while loop, conditionally determining whether the next element should be taken from S_1 or S_2 . The divide-and-conquer merge-sort algorithm is given in Code Fragment 12.2.

We illustrate a step of the merge process in Figure 12.5. During the process, index i represents the number of elements of S_1 that have been copied to S , while index j represents the number of elements of S_2 that have been copied to S . Assuming S_1 and S_2 both have at least one uncopied element, we copy the smaller of the two elements being considered. Since $i + j$ objects have been previously copied, the next element is placed in $S[i + j]$. (For example, when $i + j$ is 0, the next element is copied to $S[0]$). If we reach the end of one of the sequences, we must copy the next element from the other.

```

1 def merge(S1, S2, S):
2     """ Merge two sorted Python lists S1 and S2 into properly sized list S. """
3     i = j = 0
4     while i + j < len(S):
5         if j == len(S2) or (i < len(S1) and S1[i] < S2[j]):
6             S[i+j] = S1[i]                # copy ith element of S1 as next item of S
7             i += 1
8         else:
9             S[i+j] = S2[j]                # copy jth element of S2 as next item of S
10            j += 1

```

Code Fragment 12.1: An implementation of the merge operation for Python's array-based list class.

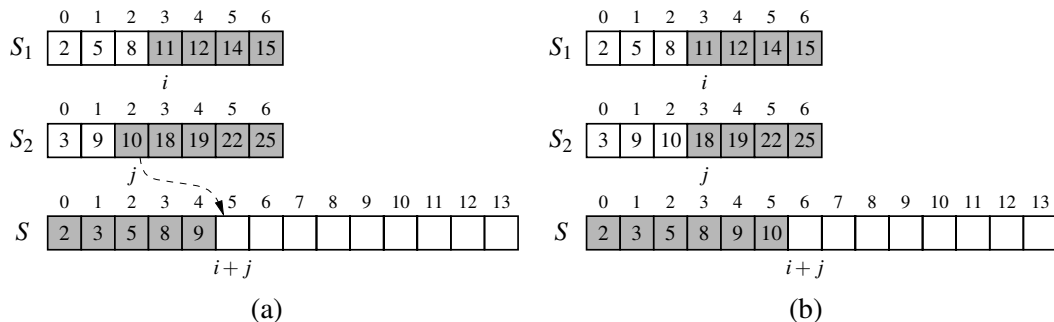


Figure 12.5: A step in the merge of two sorted arrays for which $S_2[j] < S_1[i]$. We show the arrays before the copy step in (a) and after it in (b).

```

1 def merge_sort(S):
2     """Sort the elements of Python list S using the merge-sort algorithm."""
3     n = len(S)
4     if n < 2:
5         return                # list is already sorted
6     # divide
7     mid = n // 2
8     S1 = S[0:mid]              # copy of first half
9     S2 = S[mid:n]              # copy of second half
10    # conquer (with recursion)
11    merge_sort(S1)              # sort copy of first half
12    merge_sort(S2)              # sort copy of second half
13    # merge results
14    merge(S1, S2, S)            # merge sorted halves back into S

```

Code Fragment 12.2: An implementation of the recursive merge-sort algorithm for Python’s array-based list class (using the merge function defined in Code Fragment 12.1).

12.2.3 The Running Time of Merge-Sort

We begin by analyzing the running time of the merge algorithm. Let n_1 and n_2 be the number of elements of S_1 and S_2 , respectively. It is clear that the operations performed inside each pass of the while loop take $O(1)$ time. The key observation is that during each iteration of the loop, one element is copied from either S_1 or S_2 into S (and that element is considered no further). Therefore, the number of iterations of the loop is $n_1 + n_2$. Thus, the running time of algorithm merge is $O(n_1 + n_2)$.

Having analyzed the running time of the merge algorithm used to combine subproblems, let us analyze the running time of the entire merge-sort algorithm, assuming it is given an input sequence of n elements. For simplicity, we restrict our attention to the case where n is a power of 2. We leave it to an exercise (R-12.3) to show that the result of our analysis also holds when n is not a power of 2.

When evaluating the merge-sort recursion, we rely on the analysis technique introduced in Section 4.2. We account for the amount of time spent within each recursive call, but excluding any time spent waiting for successive recursive calls to terminate. In the case of our merge_sort function, we account for the time to divide the sequence into two subsequences, and the call to merge to combine the two sorted sequences, but we exclude the two recursive calls to merge_sort.

A merge-sort tree T , as portrayed in Figures 12.2 through 12.4, can guide our analysis. Consider a recursive call associated with a node v of the merge-sort tree T . The divide step at node v is straightforward; this step runs in time proportional to the size of the sequence for v , based on the use of slicing to create copies of the two list halves. We have already observed that the merging step also takes time that is linear in the size of the merged sequence. If we let i denote the depth of node v , the time spent at node v is $O(n/2^i)$, since the size of the sequence handled by the recursive call associated with v is equal to $n/2^i$.

Looking at the tree T more globally, as shown in Figure 12.6, we see that, given our definition of “time spent at a node,” the running time of merge-sort is equal to the sum of the times spent at the nodes of T . Observe that T has exactly 2^i nodes at depth i . This simple observation has an important consequence, for it implies that the overall time spent at all the nodes of T at depth i is $O(2^i \cdot n/2^i)$, which is $O(n)$. By Proposition 12.1, the height of T is $\lceil \log n \rceil$. Thus, since the time spent at each of the $\lceil \log n \rceil + 1$ levels of T is $O(n)$, we have the following result:

Proposition 12.2: *Algorithm merge-sort sorts a sequence S of size n in $O(n \log n)$ time, assuming two elements of S can be compared in $O(1)$ time.*

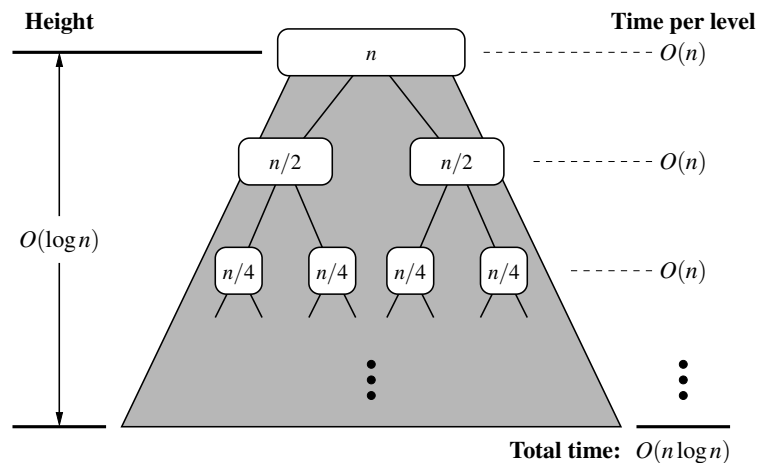


Figure 12.6: A visual analysis of the running time of merge-sort. Each node represents the time spent in a particular recursive call, labeled with the size of its subproblem.

12.2.4 Merge-Sort and Recurrence Equations ★

There is another way to justify that the running time of the merge-sort algorithm is $O(n \log n)$ (Proposition 12.2). Namely, we can deal more directly with the recursive nature of the merge-sort algorithm. In this section, we present such an analysis of the running time of merge-sort, and in so doing, introduce the mathematical concept of a **recurrence equation** (also known as **recurrence relation**).

Let the function $t(n)$ denote the worst-case running time of merge-sort on an input sequence of size n . Since merge-sort is recursive, we can characterize function $t(n)$ by means of an equation where the function $t(n)$ is recursively expressed in terms of itself. In order to simplify our characterization of $t(n)$, let us restrict our attention to the case when n is a power of 2. (We leave the problem of showing that our asymptotic characterization still holds in the general case as an exercise.) In this case, we can specify the definition of $t(n)$ as

$$t(n) = \begin{cases} b & \text{if } n \leq 1 \\ 2t(n/2) + cn & \text{otherwise.} \end{cases}$$

An expression such as the one above is called a recurrence equation, since the function appears on both the left- and right-hand sides of the equal sign. Although such a characterization is correct and accurate, what we really desire is a big-Oh type of characterization of $t(n)$ that does not involve the function $t(n)$ itself. That is, we want a **closed-form** characterization of $t(n)$.

We can obtain a closed-form solution by applying the definition of a recurrence equation, assuming n is relatively large. For example, after one more application of the equation above, we can write a new recurrence for $t(n)$ as

$$\begin{aligned} t(n) &= 2(2t(n/2^2) + (cn/2)) + cn \\ &= 2^2t(n/2^2) + 2(cn/2) + cn = 2^2t(n/2^2) + 2cn. \end{aligned}$$

If we apply the equation again, we get $t(n) = 2^3t(n/2^3) + 3cn$. At this point, we should see a pattern emerging, so that after applying this equation i times, we get

$$t(n) = 2^i t(n/2^i) + icn.$$

The issue that remains, then, is to determine when to stop this process. To see when to stop, recall that we switch to the closed form $t(n) = b$ when $n \leq 1$, which will occur when $2^i = n$. In other words, this will occur when $i = \log n$. Making this substitution, then, yields

$$\begin{aligned} t(n) &= 2^{\log n} t(n/2^{\log n}) + (\log n)cn \\ &= nt(1) + cn \log n \\ &= nb + cn \log n. \end{aligned}$$

That is, we get an alternative justification of the fact that $t(n)$ is $O(n \log n)$.

12.2.5 Alternative Implementations of Merge-Sort

Sorting Linked Lists

The merge-sort algorithm can easily be adapted to use any form of a basic queue as its container type. In Code Fragment 12.3, we provide such an implementation, based on use of the `LinkedQueue` class from Section 7.1.2. The $O(n \log n)$ bound for merge-sort from Proposition 12.2 applies to this implementation as well, since each basic operation runs in $O(1)$ time when implemented with a linked list. We show an example execution of this version of the merge algorithm in Figure 12.7.

```

1  def merge(S1, S2, S):
2      """ Merge two sorted queue instances S1 and S2 into empty queue S."""
3      while not S1.is_empty() and not S2.is_empty():
4          if S1.first() < S2.first():
5              S.enqueue(S1.dequeue())
6          else:
7              S.enqueue(S2.dequeue())
8      while not S1.is_empty():           # move remaining elements of S1 to S
9          S.enqueue(S1.dequeue())
10     while not S2.is_empty():           # move remaining elements of S2 to S
11         S.enqueue(S2.dequeue())
12
13  def merge_sort(S):
14      """ Sort the elements of queue S using the merge-sort algorithm."""
15      n = len(S)
16      if n < 2:
17          return                        # list is already sorted
18      # divide
19      S1 = LinkedQueue()                # or any other queue implementation
20      S2 = LinkedQueue()
21      while len(S1) < n // 2:           # move the first n//2 elements to S1
22          S1.enqueue(S.dequeue())
23      while not S.is_empty():           # move the rest to S2
24          S2.enqueue(S.dequeue())
25      # conquer (with recursion)
26      merge_sort(S1)                    # sort first half
27      merge_sort(S2)                    # sort second half
28      # merge results
29      merge(S1, S2, S)                  # merge sorted halves back into S

```

Code Fragment 12.3: An implementation of merge-sort using a basic queue.

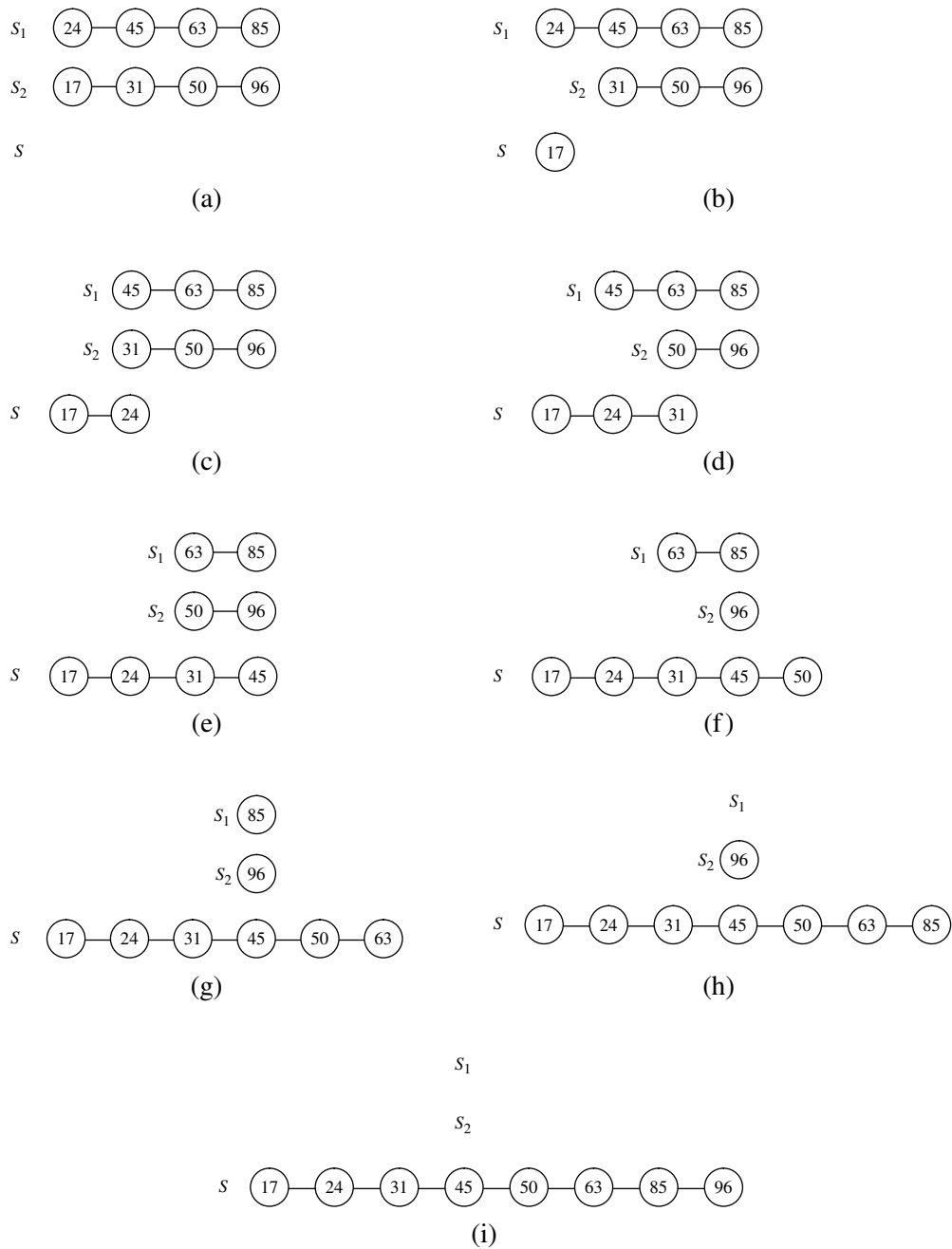


Figure 12.7: Example of an execution of the merge algorithm, as implemented in Code Fragment 12.3 using queues.

A Bottom-Up (Nonrecursive) Merge-Sort

There is a nonrecursive version of array-based merge-sort, which runs in $O(n \log n)$ time. It is a bit faster than recursive merge-sort in practice, as it avoids the extra overheads of recursive calls and temporary memory at each level. The main idea is to perform merge-sort bottom-up, performing the merges level by level going up the merge-sort tree. Given an input array of elements, we begin by merging every successive pair of elements into sorted runs of length two. We merge these runs into runs of length four, merge these new runs into runs of length eight, and so on, until the array is sorted. To keep the space usage reasonable, we deploy a second array that stores the merged runs (swapping input and output arrays after each iteration). We give a Python implementation in Code Fragment 12.4. A similar bottom-up approach can be used for sorting linked lists. (See Exercise C-12.29.)

```

1  def merge(src, result, start, inc):
2      """ Merge src[start:start+inc] and src[start+inc:start+2*inc] into result."""
3      end1 = start+inc                # boundary for run 1
4      end2 = min(start+2*inc, len(src)) # boundary for run 2
5      x, y, z = start, start+inc, start # index into run 1, run 2, result
6      while x < end1 and y < end2:
7          if src[x] < src[y]:
8              result[z] = src[x]; x += 1 # copy from run 1 and increment x
9          else:
10             result[z] = src[y]; y += 1 # copy from run 2 and increment y
11             z += 1                     # increment z to reflect new result
12     if x < end1:
13         result[z:end2] = src[x:end1] # copy remainder of run 1 to output
14     elif y < end2:
15         result[z:end2] = src[y:end2] # copy remainder of run 2 to output
16
17 def merge_sort(S):
18     """Sort the elements of Python list S using the merge-sort algorithm."""
19     n = len(S)
20     logn = math.ceil(math.log(n,2))
21     src, dest = S, [None] * n # make temporary storage for dest
22     for i in (2**k for k in range(logn)): # pass i creates all runs of length 2i
23         for j in range(0, n, 2*i): # each pass merges two length i runs
24             merge(src, dest, j, i)
25         src, dest = dest, src # reverse roles of lists
26     if S is not src:
27         S[0:n] = src[0:n] # additional copy to get results to S

```

Code Fragment 12.4: An implementation of the nonrecursive merge-sort algorithm.

12.3 Quick-Sort

The next sorting algorithm we discuss is called *quick-sort*. Like merge-sort, this algorithm is also based on the *divide-and-conquer* paradigm, but it uses this technique in a somewhat opposite manner, as all the hard work is done *before* the recursive calls.

High-Level Description of Quick-Sort

The quick-sort algorithm sorts a sequence S using a simple recursive approach. The main idea is to apply the divide-and-conquer technique, whereby we divide S into subsequences, recur to sort each subsequence, and then combine the sorted subsequences by a simple concatenation. In particular, the quick-sort algorithm consists of the following three steps (see Figure 12.8):

1. **Divide:** If S has at least two elements (nothing needs to be done if S has zero or one element), select a specific element x from S , which is called the *pivot*. As is common practice, choose the pivot x to be the last element in S . Remove all the elements from S and put them into three sequences:

- L , storing the elements in S less than x
- E , storing the elements in S equal to x
- G , storing the elements in S greater than x

Of course, if the elements of S are distinct, then E holds just one element—the pivot itself.

2. **Conquer:** Recursively sort sequences L and G .
3. **Combine:** Put back the elements into S in order by first inserting the elements of L , then those of E , and finally those of G .

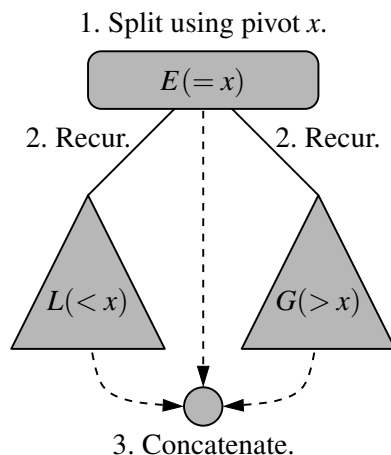


Figure 12.8: A visual schematic of the quick-sort algorithm.

Like merge-sort, the execution of quick-sort can be visualized by means of a binary recursion tree, called the *quick-sort tree*. Figure 12.9 summarizes an execution of the quick-sort algorithm by showing the input and output sequences processed at each node of the quick-sort tree. The step-by-step evolution of the quick-sort tree is shown in Figures 12.10, 12.11, and 12.12.

Unlike merge-sort, however, the height of the quick-sort tree associated with an execution of quick-sort is linear in the worst case. This happens, for example, if the sequence consists of n distinct elements and is already sorted. Indeed, in this case, the standard choice of the last element as pivot yields a subsequence L of size $n - 1$, while subsequence E has size 1 and subsequence G has size 0. At each invocation of quick-sort on subsequence L , the size decreases by 1. Hence, the height of the quick-sort tree is $n - 1$.

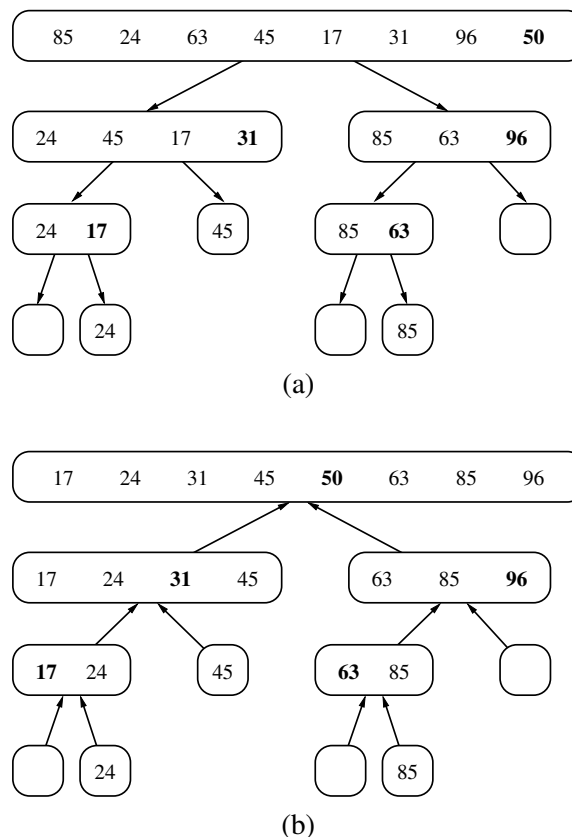


Figure 12.9: Quick-sort tree T for an execution of the quick-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T ; (b) output sequences generated at each node of T . The pivot used at each level of the recursion is shown in bold.

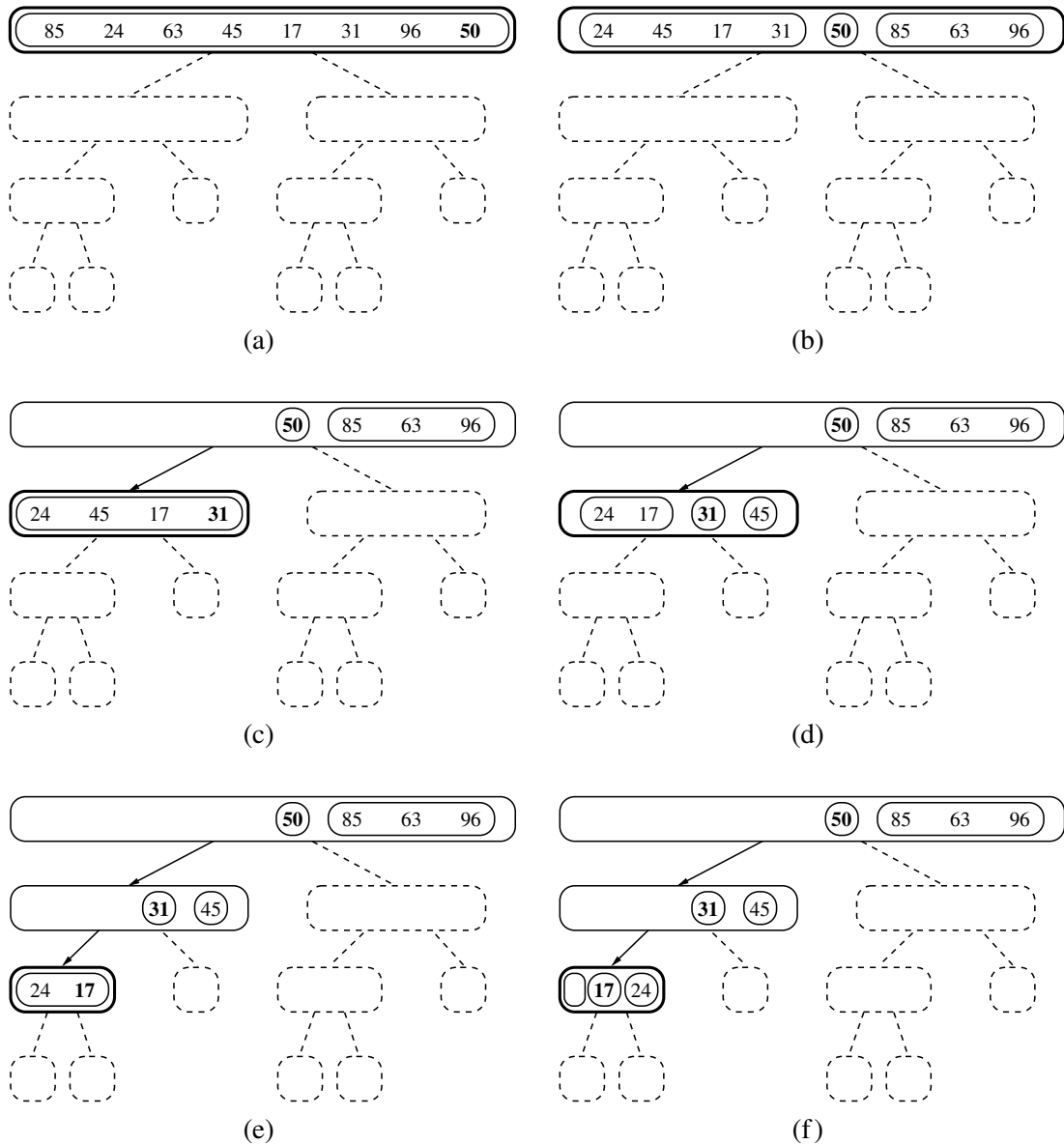


Figure 12.10: Visualization of quick-sort. Each node of the tree represents a recursive call. The nodes drawn with dashed lines represent calls that have not been made yet. The node drawn with thick lines represents the running invocation. The empty nodes drawn with thin lines represent terminated calls. The remaining nodes represent suspended calls (that is, active invocations that are waiting for a child invocation to return). Note the divide steps performed in (b), (d), and (f). (Continues in Figure 12.11.)

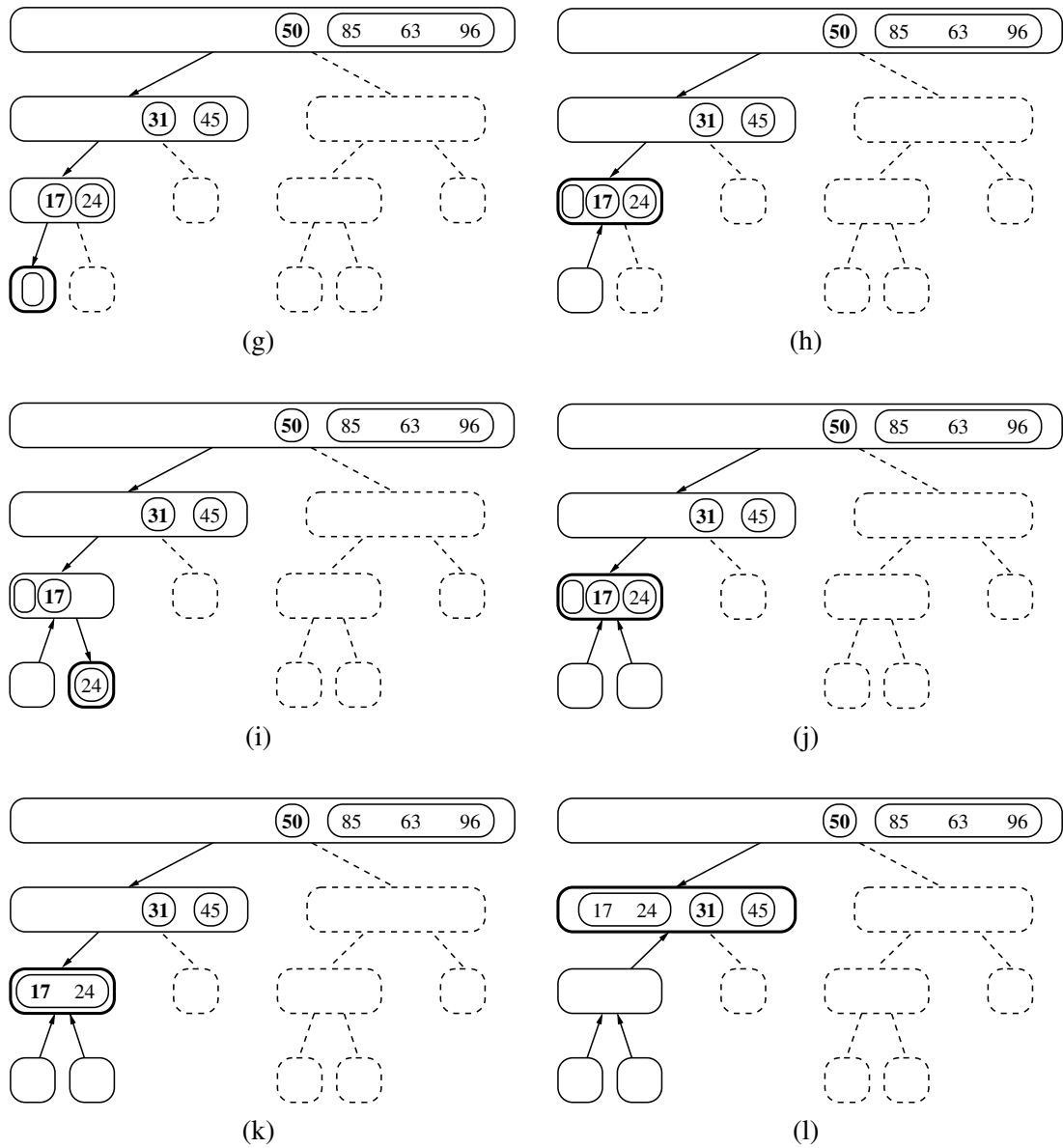


Figure 12.11: Visualization of an execution of quick-sort. Note the concatenation step performed in (k). (Continues in Figure 12.12.)

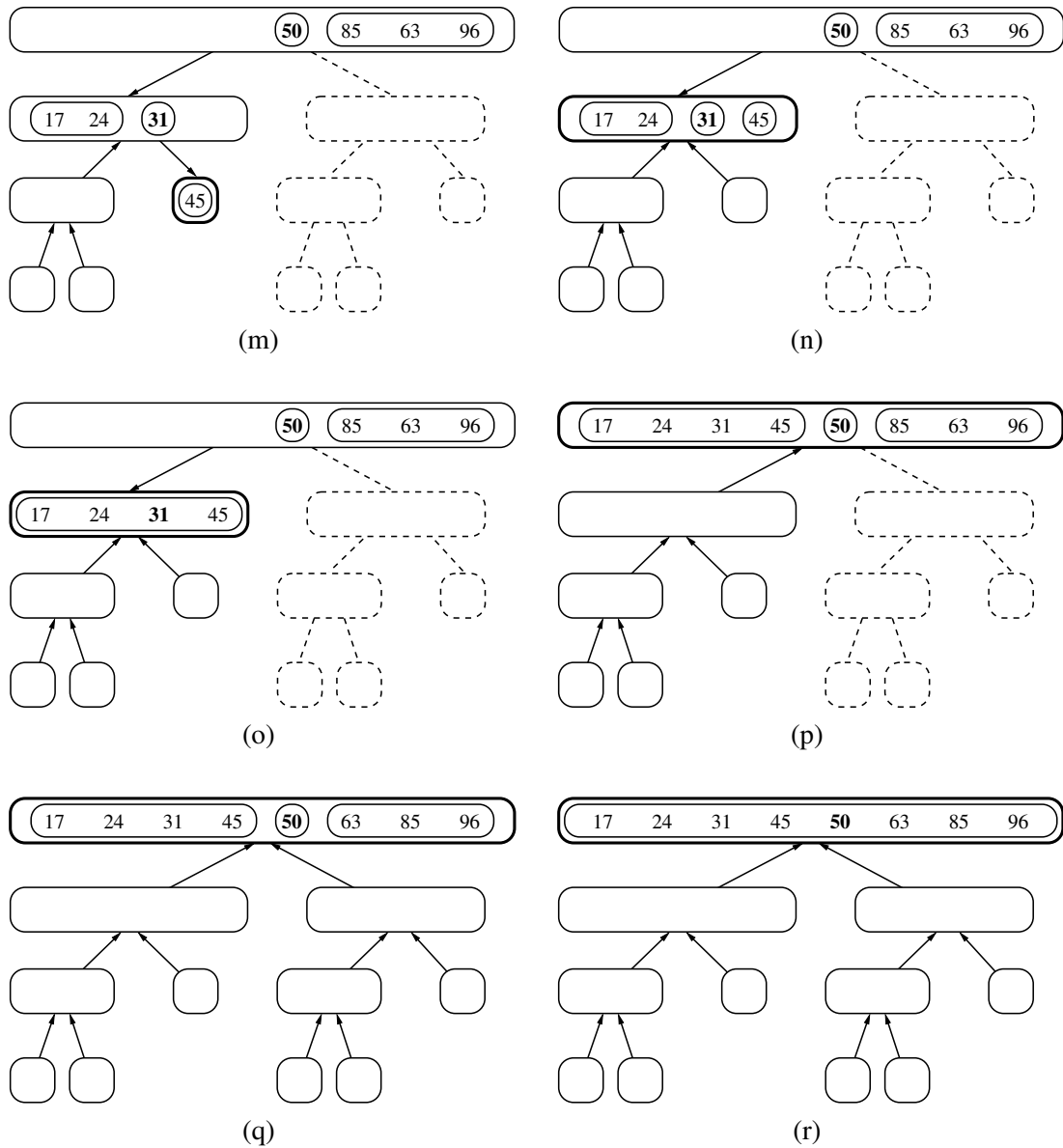


Figure 12.12: Visualization of an execution of quick-sort. Several invocations between (p) and (q) have been omitted. Note the concatenation steps performed in (o) and (r). (Continued from Figure 12.11.)

Performing Quick-Sort on General Sequences

In Code Fragment 12.5, we give an implementation of the quick-sort algorithm that works on any sequence type that operates as a queue. This particular version relies on the `LinkedQueue` class from Section 7.1.2; we provide a more streamlined implementation of quick-sort using an array-based sequence in Section 12.3.2.

Our implementation chooses the first item of the queue as the pivot (since it is easily accessible), and then it divides sequence S into queues L , E , and G of elements that are respectively less than, equal to, and greater than the pivot. We then recur on the L and G lists, and transfer elements from the sorted lists L , E , and G back to S . All of the queue operations run in $O(1)$ worst-case time when implemented with a linked list.

```

1  def quick_sort(S):
2      """Sort the elements of queue S using the quick-sort algorithm."""
3      n = len(S)
4      if n < 2:
5          return                                # list is already sorted
6      # divide
7      p = S.first( )                            # using first as arbitrary pivot
8      L = LinkedQueue()
9      E = LinkedQueue()
10     G = LinkedQueue()
11     while not S.is_empty():                    # divide S into L, E, and G
12         if S.first( ) < p:
13             L.enqueue(S.dequeue())
14         elif p < S.first():
15             G.enqueue(S.dequeue())
16         else:                                  # S.first() must equal pivot
17             E.enqueue(S.dequeue())
18     # conquer (with recursion)
19     quick_sort(L)                             # sort elements less than p
20     quick_sort(G)                             # sort elements greater than p
21     # concatenate results
22     while not L.is_empty():
23         S.enqueue(L.dequeue())
24     while not E.is_empty():
25         S.enqueue(E.dequeue())
26     while not G.is_empty():
27         S.enqueue(G.dequeue())

```

Code Fragment 12.5: Quick-sort for a sequence S implemented as a queue.

Running Time of Quick-Sort

We can analyze the running time of quick-sort with the same technique used for merge-sort in Section 12.2.3. Namely, we can identify the time spent at each node of the quick-sort tree T and sum up the running times for all the nodes.

Examining Code Fragment 12.5, we see that the divide step and the final concatenation of quick-sort can be implemented in linear time. Thus, the time spent at a node v of T is proportional to the *input size* $s(v)$ of v , defined as the size of the sequence handled by the invocation of quick-sort associated with node v . Since subsequence E has at least one element (the pivot), the sum of the input sizes of the children of v is at most $s(v) - 1$.

Let s_i denote the sum of the input sizes of the nodes at depth i for a particular quick-sort tree T . Clearly, $s_0 = n$, since the root r of T is associated with the entire sequence. Also, $s_1 \leq n - 1$, since the pivot is not propagated to the children of r . More generally, it must be that $s_i < s_{i-1}$ since the elements of the subsequences at depth i all come from distinct subsequences at depth $i - 1$, and at least one element from depth $i - 1$ does not propagate to depth i because it is in a set E (in fact, one element from *each node* at depth $i - 1$ does not propagate to depth i).

We can therefore bound the overall running time of an execution of quick-sort as $O(n \cdot h)$ where h is the overall height of the quick-sort tree T for that execution. Unfortunately, in the worst case, the height of a quick-sort tree is $\Theta(n)$, as observed in Section 12.3. Thus, quick-sort runs in $O(n^2)$ worst-case time. Paradoxically, if we choose the pivot as the last element of the sequence, this worst-case behavior occurs for problem instances when sorting should be easy—if the sequence is already sorted.

Given its name, we would expect quick-sort to run quickly, and it often does in practice. The best case for quick-sort on a sequence of distinct elements occurs when subsequences L and G have roughly the same size. In that case, as we saw with merge-sort, the tree has height $O(\log n)$ and therefore quick-sort runs in $O(n \log n)$ time; we leave the justification of this fact as an exercise (R-12.10). More so, we can observe an $O(n \log n)$ running time even if the split between L and G is not as perfect. For example, if every divide step caused one subsequence to have one-fourth of those elements and the other to have three-fourths of the elements, the height of the tree would remain $O(\log n)$ and thus the overall performance $O(n \log n)$.

We will see in the next section that introducing randomization in the choice of a pivot will make quick-sort essentially behave in this way on average, with an expected running time that is $O(n \log n)$.

12.3.1 Randomized Quick-Sort

One common method for analyzing quick-sort is to assume that the pivot will always divide the sequence in a reasonably balanced manner. We feel such an assumption would presuppose knowledge about the input distribution that is typically not available, however. For example, we would have to assume that we will rarely be given “almost” sorted sequences to sort, which are actually common in many applications. Fortunately, this assumption is not needed in order for us to match our intuition to quick-sort’s behavior.

In general, we desire some way of getting close to the best-case running time for quick-sort. The way to get close to the best-case running time, of course, is for the pivot to divide the input sequence S almost equally. If this outcome were to occur, then it would result in a running time that is asymptotically the same as the best-case running time. That is, having pivots close to the “middle” of the set of elements leads to an $O(n \log n)$ running time for quick-sort.

Picking Pivots at Random

Since the goal of the partition step of the quick-sort method is to divide the sequence S with sufficient balance, let us introduce randomization into the algorithm and pick as the pivot a **random element** of the input sequence. That is, instead of picking the pivot as the first or last element of S , we pick an element of S at random as the pivot, keeping the rest of the algorithm unchanged. This variation of quick-sort is called **randomized quick-sort**. The following proposition shows that the expected running time of randomized quick-sort on a sequence with n elements is $O(n \log n)$. This expectation is taken over all the possible random choices the algorithm makes, and is independent of any assumptions about the distribution of the possible input sequences the algorithm is likely to be given.

Proposition 12.3: *The expected running time of randomized quick-sort on a sequence S of size n is $O(n \log n)$.*

Justification: We assume two elements of S can be compared in $O(1)$ time. Consider a single recursive call of randomized quick-sort, and let n denote the size of the input for this call. Say that this call is “good” if the pivot chosen is such that subsequences L and G have size at least $n/4$ and at most $3n/4$ each; otherwise, a call is “bad.”

Now, consider the implications of our choosing a pivot uniformly at random. Note that there are $n/2$ possible good choices for the pivot for any given call of size n of the randomized quick-sort algorithm. Thus, the probability that any call is good is $1/2$. Note further that a good call will at least partition a list of size n into two lists of size $3n/4$ and $n/4$, and a bad call could be as bad as producing a single call of size $n - 1$.

Now consider a recursion trace for randomized quick-sort. This trace defines a binary tree, T , such that each node in T corresponds to a different recursive call on a subproblem of sorting a portion of the original list.

Say that a node v in T is in **size group** i if the size of v 's subproblem is greater than $(3/4)^{i+1}n$ and at most $(3/4)^i n$. Let us analyze the expected time spent working on all the subproblems for nodes in size group i . By the linearity of expectation (Proposition B.19), the expected time for working on all these subproblems is the sum of the expected times for each one. Some of these nodes correspond to good calls and some correspond to bad calls. But note that, since a good call occurs with probability $1/2$, the expected number of consecutive calls we have to make before getting a good call is 2. Moreover, notice that as soon as we have a good call for a node in size group i , its children will be in size groups higher than i . Thus, for any element x from in the input list, the expected number of nodes in size group i containing x in their subproblems is 2. In other words, the expected total size of all the subproblems in size group i is $2n$. Since the nonrecursive work we perform for any subproblem is proportional to its size, this implies that the total expected time spent processing subproblems for nodes in size group i is $O(n)$.

The number of size groups is $\log_{4/3} n$, since repeatedly multiplying by $3/4$ is the same as repeatedly dividing by $4/3$. That is, the number of size groups is $O(\log n)$. Therefore, the total expected running time of randomized quick-sort is $O(n \log n)$. (See Figure 12.13.) ■

If fact, we can show that the running time of randomized quick-sort is $O(n \log n)$ with high probability. (See Exercise C-12.54.)

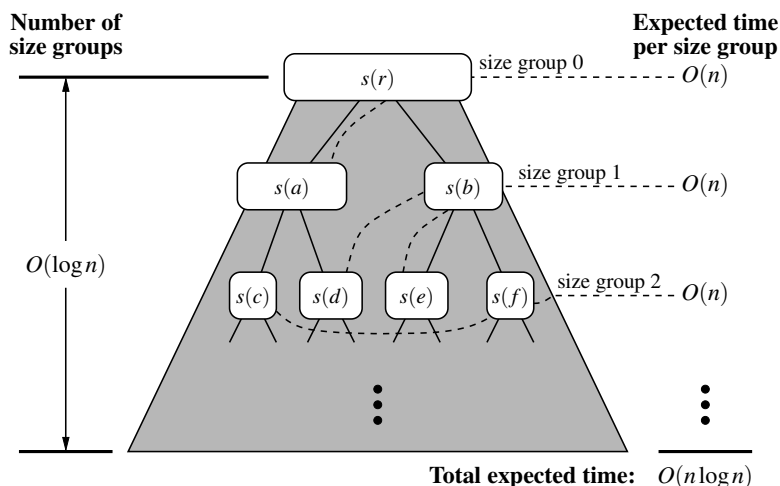


Figure 12.13: A visual time analysis of the quick-sort tree T . Each node is shown labeled with the size of its subproblem.

12.3.2 Additional Optimizations for Quick-Sort

An algorithm is *in-place* if it uses only a small amount of memory in addition to that needed for the original input. Our implementation of heap-sort, from Section 9.4.2, is an example of such an in-place sorting algorithm. Our implementation of quick-sort from Code Fragment 12.5 does not qualify as in-place because we use additional containers L , E , and G when dividing a sequence S within each recursive call. Quick-sort of an array-based sequence can be adapted to be in-place, and such an optimization is used in most deployed implementations.

Performing the quick-sort algorithm in-place requires a bit of ingenuity, however, for we must use the input sequence itself to store the subsequences for all the recursive calls. We show algorithm `inplace_quick_sort`, which performs in-place quick-sort, in Code Fragment 12.6. Our implementation assumes that the input sequence, S , is given as a Python list of elements. In-place quick-sort modifies the input sequence using element swapping and does not explicitly create subsequences. Instead, a subsequence of the input sequence is implicitly represented by a range of positions specified by a leftmost index a and a rightmost index b . The

```

1  def inplace_quick_sort(S, a, b):
2      """Sort the list from S[a] to S[b] inclusive using the quick-sort algorithm."""
3      if a >= b: return                # range is trivially sorted
4      pivot = S[b]                    # last element of range is pivot
5      left = a                        # will scan rightward
6      right = b-1                    # will scan leftward
7      while left <= right:
8          # scan until reaching value equal or larger than pivot (or right marker)
9          while left <= right and S[left] < pivot:
10             left += 1
11         # scan until reaching value equal or smaller than pivot (or left marker)
12         while left <= right and pivot < S[right]:
13             right -= 1
14         if left <= right:             # scans did not strictly cross
15             S[left], S[right] = S[right], S[left]    # swap values
16             left, right = left + 1, right - 1        # shrink range
17
18     # put pivot into its final place (currently marked by left index)
19     S[left], S[b] = S[b], S[left]
20     # make recursive calls
21     inplace_quick_sort(S, a, left - 1)
22     inplace_quick_sort(S, left + 1, b)

```

Code Fragment 12.6: In-place quick-sort for a Python list S .

divide step is performed by scanning the array simultaneously using local variables l , which advances forward, and r , which advances backward, swapping pairs of elements that are in reverse order, as shown in Figure 12.14. When these two indices pass each other, the division step is complete and the algorithm completes by recurring on these two sublists. There is no explicit “combine” step, because the concatenation of the two sublists is implicit to the in-place use of the original list.

It is worth noting that if a sequence has duplicate values, we are not explicitly creating three sublists L , E , and G , as in our original quick-sort description. We instead allow elements equal to the pivot (other than the pivot itself) to be dispersed across the two sublists. Exercise R-12.11 explores the subtlety of our implementation in the presence of duplicate keys, and Exercise C-12.33 describes an in-place algorithm that strictly partitions into three sublists L , E , and G .

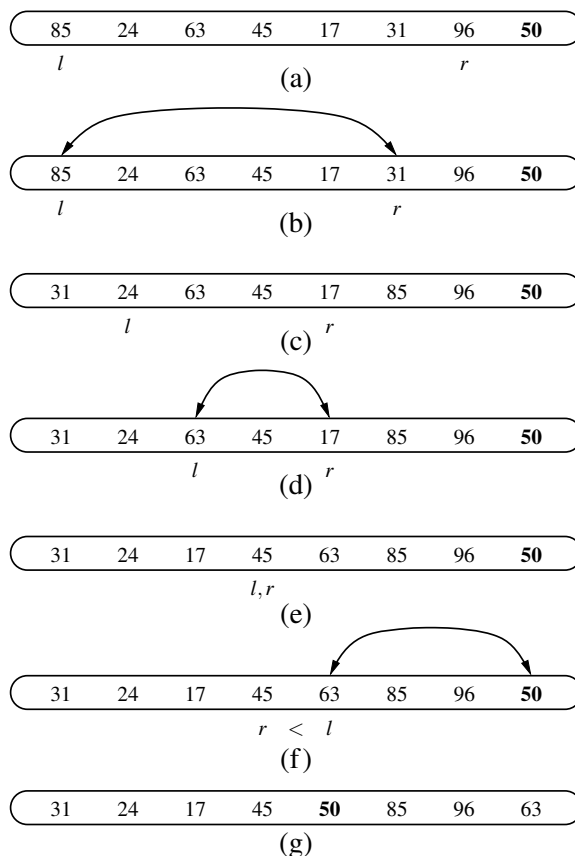


Figure 12.14: Divide step of in-place quick-sort, using index l as shorthand for identifier left, and index r as shorthand for identifier right. Index l scans the sequence from left to right, and index r scans the sequence from right to left. A swap is performed when l is at an element as large as the pivot and r is at an element as small as the pivot. A final swap with the pivot, in part (f), completes the divide step.

Although the implementation we describe in this section for dividing the sequence into two pieces is in-place, we note that the complete quick-sort algorithm needs space for a stack proportional to the depth of the recursion tree, which in this case can be as large as $n - 1$. Admittedly, the expected stack depth is $O(\log n)$, which is small compared to n . Nevertheless, a simple trick lets us guarantee the stack size is $O(\log n)$. The main idea is to design a nonrecursive version of in-place quick-sort using an explicit stack to iteratively process subproblems (each of which can be represented with a pair of indices marking subarray boundaries). Each iteration involves popping the top subproblem, splitting it in two (if it is big enough), and pushing the two new subproblems. The trick is that when pushing the new subproblems, we should first push the larger subproblem and then the smaller one. In this way, the sizes of the subproblems will at least double as we go down the stack; hence, the stack can have depth at most $O(\log n)$. We leave the details of this implementation as an exercise (P-12.56).

Pivot Selection

Our implementation in this section blindly picks the last element as the pivot at each level of the quick-sort recursion. This leaves it susceptible to the $\Theta(n^2)$ -time worst case, most notably when the original sequence is already sorted, reverse sorted, or nearly sorted.

As described in Section 12.3.1, this can be improved upon by using a randomly chosen pivot for each partition step. In practice, another common technique for choosing a pivot is to use the median of three values, taken respectively from the front, middle, and tail of the array. This *median-of-three* heuristic will more often choose a good pivot and computing a median of three may require lower overhead than selecting a pivot with a random number generator. For larger data sets, the median of more than three potential pivots might be computed.

Hybrid Approaches

Although quick-sort has very good performance on large data sets, it has rather high overhead on relatively small data sets. For example, the process of quick-sorting a sequence of eight elements, as illustrated in Figures 12.10 through 12.12, involves considerable bookkeeping. In practice, a simple algorithm like insertion-sort (Section 7.5) will execute faster when sorting such a short sequence.

It is therefore common, in optimized sorting implementations, to use a hybrid approach, with a divide-and-conquer algorithm used until the size of a subsequence falls below some threshold (perhaps 50 elements); insertion-sort can be directly invoked upon portions with length below the threshold. We will further discuss such practical considerations in Section 12.5, when comparing the performance of various sorting algorithms.

12.4 Studying Sorting through an Algorithmic Lens

Recapping our discussions on sorting to this point, we have described several methods with either a worst case or expected running time of $O(n \log n)$ on an input sequence of size n . These methods include merge-sort and quick-sort, described in this chapter, as well as heap-sort (Section 9.4.2). In this section, we study sorting as an algorithmic problem, addressing general issues about sorting algorithms.

12.4.1 Lower Bound for Sorting

A natural first question to ask is whether we can sort any faster than $O(n \log n)$ time. Interestingly, if the computational primitive used by a sorting algorithm is the comparison of two elements, this is in fact the best we can do—comparison-based sorting has an $\Omega(n \log n)$ worst-case lower bound on its running time. (Recall the notation $\Omega(\cdot)$ from Section 3.3.1.) To focus on the main cost of comparison-based sorting, let us only count comparisons, for the sake of a lower bound.

Suppose we are given a sequence $S = (x_0, x_1, \dots, x_{n-1})$ that we wish to sort, and assume that all the elements of S are distinct (this is not really a restriction since we are deriving a lower bound). We do not care if S is implemented as an array or a linked list, for the sake of our lower bound, since we are only counting comparisons. Each time a sorting algorithm compares two elements x_i and x_j (that is, it asks, “is $x_i < x_j$?”), there are two outcomes: “yes” or “no.” Based on the result of this comparison, the sorting algorithm may perform some internal calculations (which we are not counting here) and will eventually perform another comparison between two other elements of S , which again will have two outcomes. Therefore, we can represent a comparison-based sorting algorithm with a decision tree T (recall Example 8.6). That is, each internal node v in T corresponds to a comparison and the edges from position v to its children correspond to the computations resulting from either a “yes” or “no” answer. It is important to note that the hypothetical sorting algorithm in question probably has no explicit knowledge of the tree T . The tree simply represents all the possible sequences of comparisons that a sorting algorithm might make, starting from the first comparison (associated with the root) and ending with the last comparison (associated with the parent of an external node).

Each possible initial order, or *permutation*, of the elements in S will cause our hypothetical sorting algorithm to execute a series of comparisons, traversing a path in T from the root to some external node. Let us associate with each external node v in T , then, the set of permutations of S that cause our sorting algorithm to end up in v . The most important observation in our lower-bound argument is that each external node v in T can represent the sequence of comparisons for at most one permutation of S . The justification for this claim is simple: If two different

permutations P_1 and P_2 of S are associated with the same external node, then there are at least two objects x_i and x_j , such that x_i is before x_j in P_1 but x_i is after x_j in P_2 . At the same time, the output associated with v must be a specific reordering of S , with either x_i or x_j appearing before the other. But if P_1 and P_2 both cause the sorting algorithm to output the elements of S in this order, then that implies there is a way to trick the algorithm into outputting x_i and x_j in the wrong order. Since this cannot be allowed by a correct sorting algorithm, each external node of T must be associated with exactly one permutation of S . We use this property of the decision tree associated with a sorting algorithm to prove the following result:

Proposition 12.4: *The running time of any comparison-based algorithm for sorting an n -element sequence is $\Omega(n \log n)$ in the worst case.*

Justification: The running time of a comparison-based sorting algorithm must be greater than or equal to the height of the decision tree T associated with this algorithm, as described above. (See Figure 12.15.) By the argument above, each external node in T must be associated with one permutation of S . Moreover, each permutation of S must result in a different external node of T . The number of permutations of n objects is $n! = n(n-1)(n-2) \cdots 2 \cdot 1$. Thus, T must have at least $n!$ external nodes. By Proposition 8.8, the height of T is at least $\log(n!)$. This immediately justifies the proposition, because there are at least $n/2$ terms that are greater than or equal to $n/2$ in the product $n!$; hence,

$$\log(n!) \geq \log \left(\left(\frac{n}{2} \right)^{\frac{n}{2}} \right) = \frac{n}{2} \log \frac{n}{2},$$

which is $\Omega(n \log n)$. ■

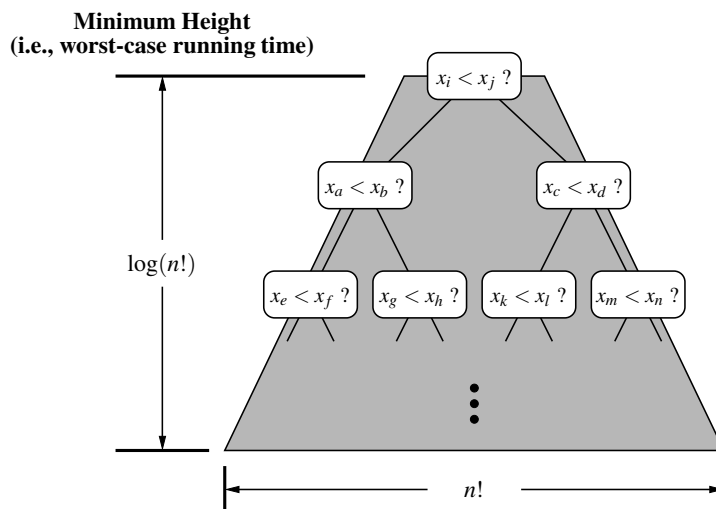


Figure 12.15: Visualizing the lower bound for comparison-based sorting.

12.4.2 Linear-Time Sorting: Bucket-Sort and Radix-Sort

In the previous section, we showed that $\Omega(n \log n)$ time is necessary, in the worst case, to sort an n -element sequence with a comparison-based sorting algorithm. A natural question to ask, then, is whether there are other kinds of sorting algorithms that can be designed to run asymptotically faster than $O(n \log n)$ time. Interestingly, such algorithms exist, but they require special assumptions about the input sequence to be sorted. Even so, such scenarios often arise in practice, such as when sorting integers from a known range or sorting character strings, so discussing them is worthwhile. In this section, we consider the problem of sorting a sequence of entries, each a key-value pair, where the keys have a restricted type.

Bucket-Sort

Consider a sequence S of n entries whose keys are integers in the range $[0, N - 1]$, for some integer $N \geq 2$, and suppose that S should be sorted according to the keys of the entries. In this case, it is possible to sort S in $O(n + N)$ time. It might seem surprising, but this implies, for example, that if N is $O(n)$, then we can sort S in $O(n)$ time. Of course, the crucial point is that, because of the restrictive assumption about the format of the elements, we can avoid using comparisons.

The main idea is to use an algorithm called **bucket-sort**, which is not based on comparisons, but on using keys as indices into a bucket array B that has cells indexed from 0 to $N - 1$. An entry with key k is placed in the “bucket” $B[k]$, which itself is a sequence (of entries with key k). After inserting each entry of the input sequence S into its bucket, we can put the entries back into S in sorted order by enumerating the contents of the buckets $B[0], B[1], \dots, B[N - 1]$ in order. We describe the bucket-sort algorithm in Code Fragment 12.7.

Algorithm bucketSort(S):

Input: Sequence S of entries with integer keys in the range $[0, N - 1]$

Output: Sequence S sorted in nondecreasing order of the keys

let B be an array of N sequences, each of which is initially empty

for each entry e in S **do**

k = the key of e

 remove e from S and insert it at the end of bucket (sequence) $B[k]$

for $i = 0$ to $N - 1$ **do**

for each entry e in sequence $B[i]$ **do**

 remove e from $B[i]$ and insert it at the end of S

Code Fragment 12.7: Bucket-sort.

It is easy to see that bucket-sort runs in $O(n + N)$ time and uses $O(n + N)$ space. Hence, bucket-sort is efficient when the range N of values for the keys is small compared to the sequence size n , say $N = O(n)$ or $N = O(n \log n)$. Still, its performance deteriorates as N grows compared to n .

An important property of the bucket-sort algorithm is that it works correctly even if there are many different elements with the same key. Indeed, we described it in a way that anticipates such occurrences.

Stable Sorting

When sorting key-value pairs, an important issue is how equal keys are handled. Let $S = ((k_0, v_0), \dots, (k_{n-1}, v_{n-1}))$ be a sequence of such entries. We say that a sorting algorithm is **stable** if, for any two entries (k_i, v_i) and (k_j, v_j) of S such that $k_i = k_j$ and (k_i, v_i) precedes (k_j, v_j) in S before sorting (that is, $i < j$), entry (k_i, v_i) also precedes entry (k_j, v_j) after sorting. Stability is important for a sorting algorithm because applications may want to preserve the initial order of elements with the same key.

Our informal description of bucket-sort in Code Fragment 12.7 guarantees stability as long as we ensure that all sequences act as queues, with elements processed and removed from the front of a sequence and inserted at the back. That is, when initially placing elements of S into buckets, we should process S from front to back, and add each element to the end of its bucket. Subsequently, when transferring elements from the buckets back to S , we should process each $B[i]$ from front to back, with those elements added to the end of S .

Radix-Sort

One of the reasons that stable sorting is so important is that it allows the bucket-sort approach to be applied to more general contexts than to sort integers. Suppose, for example, that we want to sort entries with keys that are pairs (k, l) , where k and l are integers in the range $[0, N - 1]$, for some integer $N \geq 2$. In a context such as this, it is common to define an order on these keys using the **lexicographic** (dictionary) convention, where $(k_1, l_1) < (k_2, l_2)$ if $k_1 < k_2$ or if $k_1 = k_2$ and $l_1 < l_2$ (see page 15). This is a pairwise version of the lexicographic comparison function, which can be applied to equal-length character strings, or to tuples of length d .

The **radix-sort** algorithm sorts a sequence S of entries with keys that are pairs, by applying a stable bucket-sort on the sequence twice; first using one component of the pair as the key when ordering and then using the second component. But which order is correct? Should we first sort on the k 's (the first component) and then on the l 's (the second component), or should it be the other way around?

To gain intuition before answering this question, we consider the following example.

Example 12.5: Consider the following sequence S (we show only the keys):

$$S = ((3,3), (1,5), (2,5), (1,2), (2,3), (1,7), (3,2), (2,2)).$$

If we sort S stably on the first component, then we get the sequence

$$S_1 = ((1,5), (1,2), (1,7), (2,5), (2,3), (2,2), (3,3), (3,2)).$$

If we then stably sort this sequence S_1 using the second component, we get the sequence

$$S_{1,2} = ((1,2), (2,2), (3,2), (2,3), (3,3), (1,5), (2,5), (1,7)),$$

which is unfortunately not a sorted sequence. On the other hand, if we first stably sort S using the second component, then we get the sequence

$$S_2 = ((1,2), (3,2), (2,2), (3,3), (2,3), (1,5), (2,5), (1,7)).$$

If we then stably sort sequence S_2 using the first component, we get the sequence

$$S_{2,1} = ((1,2), (1,5), (1,7), (2,2), (2,3), (2,5), (3,2), (3,3)),$$

which is indeed sequence S lexicographically ordered.

So, from this example, we are led to believe that we should first sort using the second component and then again using the first component. This intuition is exactly right. By first stably sorting by the second component and then again by the first component, we guarantee that if two entries are equal in the second sort (by the first component), then their relative order in the starting sequence (which is sorted by the second component) is preserved. Thus, the resulting sequence is guaranteed to be sorted lexicographically every time. We leave to a simple exercise (R-12.18) the determination of how this approach can be extended to triples and other d -tuples of numbers. We can summarize this section as follows:

Proposition 12.6: Let S be a sequence of n key-value pairs, each of which has a key $(k_1, k_2, \dots, k_d)$, where k_i is an integer in the range $[0, N - 1]$ for some integer $N \geq 2$. We can sort S lexicographically in time $O(d(n + N))$ using radix-sort.

Radix sort can be applied to any key that can be viewed as a composite of smaller pieces that are to be sorted lexicographically. For example, we can apply it to sort character strings of moderate length, as each individual character can be represented as an integer value. (Some care is needed to properly handle strings with varying lengths.)

12.5 Comparing Sorting Algorithms

At this point, it might be useful for us to take a moment and consider all the algorithms we have studied in this book to sort an n -element sequence.

Considering Running Time and Other Factors

We have studied several methods, such as insertion-sort, and selection-sort, that have $O(n^2)$ -time behavior in the average and worst case. We have also studied several methods with $O(n \log n)$ -time behavior, including heap-sort, merge-sort, and quick-sort. Finally, the bucket-sort and radix-sort methods run in linear time for certain types of keys. Certainly, the selection-sort algorithm is a poor choice in any application, since it runs in $O(n^2)$ time even in the best case. But, of the remaining sorting algorithms, which is the best?

As with many things in life, there is no clear “best” sorting algorithm from the remaining candidates. There are trade-offs involving efficiency, memory usage, and stability. The sorting algorithm best suited for a particular application depends on the properties of that application. In fact, the default sorting algorithm used by computing languages and systems has evolved greatly over time. We can offer some guidance and observations, therefore, based on the known properties of the “good” sorting algorithms.

Insertion-Sort

If implemented well, the running time of *insertion-sort* is $O(n + m)$, where m is the number of *inversions* (that is, the number of pairs of elements out of order). Thus, insertion-sort is an excellent algorithm for sorting small sequences (say, less than 50 elements), because insertion-sort is simple to program, and small sequences necessarily have few inversions. Also, insertion-sort is quite effective for sorting sequences that are already “almost” sorted. By “almost,” we mean that the number of inversions is small. But the $O(n^2)$ -time performance of insertion-sort makes it a poor choice outside of these special contexts.

Heap-Sort

Heap-sort, on the other hand, runs in $O(n \log n)$ time in the worst case, which is optimal for comparison-based sorting methods. Heap-sort can easily be made to execute in-place, and is a natural choice on small- and medium-sized sequences, when input data can fit into main memory. However, heap-sort tends to be outperformed by both quick-sort and merge-sort on larger sequences. A standard heap-sort does not provide a stable sort, because of the swapping of elements.

Quick-Sort

Although its $O(n^2)$ -time worst-case performance makes *quick-sort* susceptible in real-time applications where we must make guarantees on the time needed to complete a sorting operation, we expect its performance to be $O(n \log n)$ -time, and experimental studies have shown that it outperforms both heap-sort and merge-sort on many tests. Quick-sort does not naturally provide a stable sort, due to the swapping of elements during the partitioning step.

For decades quick-sort was the default choice for a general-purpose, in-memory sorting algorithm. Quick-sort was included as the `qsort` sorting utility provided in C language libraries, and was the basis for sorting utilities on Unix operating systems for many years. It was also the standard algorithm for sorting arrays in Java through version 6 of that language. (We discuss Java7 below.)

Merge-Sort

Merge-sort runs in $O(n \log n)$ time in the worst case. It is quite difficult to make merge-sort run in-place for arrays, and without that optimization the extra overhead of allocate a temporary array, and copying between the arrays is less attractive than in-place implementations of heap-sort and quick-sort for sequences that can fit entirely in a computer's main memory. Even so, merge-sort is an excellent algorithm for situations where the input is stratified across various levels of the computer's memory hierarchy (e.g., cache, main memory, external memory). In these contexts, the way that merge-sort processes runs of data in long merge streams makes the best use of all the data brought as a block into a level of memory, thereby reducing the total number of memory transfers.

The GNU sorting utility (and most current versions of the Linux operating system) relies on a multiway merge-sort variant. Since 2003, the standard sort method of Python's list class has been a hybrid approach named *Tim-sort* (designed by Tim Peters), which is essentially a bottom-up merge-sort that takes advantage of some initial runs in the data while using insertion-sort to build additional runs. Tim-sort has also become the default algorithm for sorting arrays in Java7.

Bucket-Sort and Radix-Sort

Finally, if an application involves sorting entries with small integer keys, character strings, or d -tuples of keys from a discrete range, then *bucket-sort* or *radix-sort* is an excellent choice, for it runs in $O(d(n + N))$ time, where $[0, N - 1]$ is the range of integer keys (and $d = 1$ for bucket sort). Thus, if $d(n + N)$ is significantly "below" the $n \log n$ function, then this sorting method should run faster than even quick-sort, heap-sort, or merge-sort.

12.6 Python's Built-In Sorting Functions

Python provides two built-in ways to sort data. The first is the sort method of the list class. As an example, suppose that we define the following list:

```
colors = ['red', 'green', 'blue', 'cyan', 'magenta', 'yellow']
```

That method has the effect of reordering the elements of the list into order, as defined by the natural meaning of the < operator for those elements. In the above example, within elements that are strings, the natural order is defined alphabetically. Therefore, after a call to colors.sort(), the order of the list would become:

```
['blue', 'cyan', 'green', 'magenta', 'red', 'yellow']
```

Python also supports a built-in function, named sorted, that can be used to produce a new ordered list containing the elements of any existing iterable container. Going back to our original example, the syntax sorted(colors) would return a new list of those colors, in alphabetical order, while leaving the contents of the original list unchanged. This second form is more general because it can be applied to any iterable object as a parameter; for example, sorted('green') returns ['e', 'e', 'g', 'n', 'r'].

12.6.1 Sorting According to a Key Function

There are many situations in which we wish to sort a list of elements, but according to some order other than the natural order defined by the < operator. For example, we might wish to sort a list of strings from shortest to longest (rather than alphabetically). Both of Python's built-in sort functions allow a caller to control the notion of order that is used when sorting. This is accomplished by providing, as an optional keyword parameter, a reference to a secondary function that computes a *key* for each element of the primary sequence; then the primary elements are sorted based on the natural order of their keys. (See pages 27 and 28 of Section 1.5.1 for a discussion of this technique in the context of the built-in min and max functions.)

A key function must be a one-parameter function that accepts an element as a parameter and returns a key. For example, we could use the built-in len function when sorting strings by length, as a call len(s) for string s returns its length. To sort our colors list based on length, we use the syntax colors.sort(key=len) to mutate the list or sorted(colors, key=len) to generate a new ordered list, while leaving the original alone. When sorted with the length function as a key, the contents are:

```
['red', 'blue', 'cyan', 'green', 'yellow', 'magenta']
```

These built-in functions also support a keyword parameter, reverse, that can be set to True to cause the sort order to be from largest to smallest.

Decorate-Sort-Undecorate Design Pattern

Python’s support for a key function when sorting is implemented using what is known as the *decorate-sort-undecorate design pattern*. It proceeds in 3 steps:

1. Each element of the list is temporarily replaced with a “decorated” version that includes the result of the key function applied to the element.
2. The list is sorted based upon the natural order of the keys (Figure 12.16).
3. The decorated elements are replaced by the original elements.

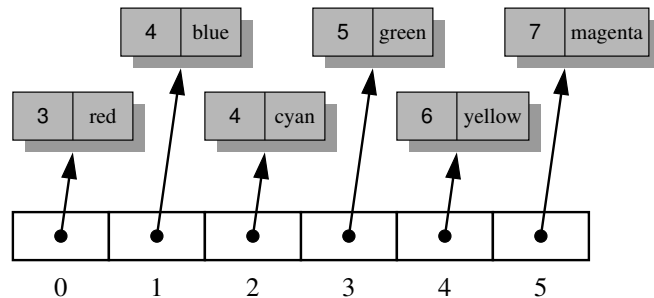


Figure 12.16: A list of “decorated” strings, using their lengths as decoration. This list has been sorted by those keys.

Although there is already built-in support for Python, if we were to implement such a strategy ourselves, a natural way to represent a “decorated” element is using the same composition strategy that we used for representing key-value pairs within a priority queue. Code Fragment 9.1 of Section 9.2.1 includes just such an `_Item` class, defined so that the `<` operator for items relies upon the given keys. With such a composition, we could trivially adapt any sorting algorithm to use the decorate-sort-undecorate pattern, as demonstrated in Code Fragment 12.8 with merge-sort.

```

1 def decorated_merge_sort(data, key=None):
2     """ Demonstration of the decorate-sort-undecorate pattern. """
3     if key is not None:
4         for j in range(len(data)):
5             data[j] = _Item(key(data[j]), data[j])    # decorate each element
6         merge_sort(data)                             # sort with existing algorithm
7     if key is not None:
8         for j in range(len(data)):
9             data[j] = data[j]._value                 # undecorate each element

```

Code Fragment 12.8: An approach for implementing the decorate-sort-undecorate pattern based upon the array-based merge-sort of Code Fragment 12.1. The `_Item` class is identical to that which was used in the `PriorityQueueBase` class. (See Code Fragment 9.1.)

Inside the CPython Timsort

Motivation and Context

Tim Peters designed Timsort in 2002 for Python's `list.sort()` (and `sorted()`).

At the time, Python was moving away from a simple, naive mergesort. That old sort worked fine for correctness and stability, but it didn't exploit a very common property of *real-world data*: partial ordering.

Many real-world lists aren't random noise. They often have big stretches that are already sorted (e.g. time series, logs, partially grouped items). Tim wanted a sort that:

- Was **stable** (preserves order of equal items)
- Ran in $O(n \log n)$ worst-case time
- Ran in $O(n)$ time on nearly sorted inputs
- Had **excellent constant factors** (i.e. fast in practice)

The key idea: *detect natural runs and exploit them*.

What is a “Run”?

A **run** is a maximal monotonically increasing (or decreasing) subsequence.

Example:

[1, 2, 3, 0, -1, -2, 5, 6]

- Run 1: 1, 2, 3 (increasing)
- Run 2: 0, -1, -2 (decreasing)
- Run 3: 5, 6 (increasing)

If you reverse decreasing runs to make them increasing, you get several sorted blocks that you can merge.

Timsort starts by scanning the input to find these natural runs. It then pushes them onto a stack and merges them in a carefully controlled way.

Why? Because real-world data often has big runs already. Instead of ignoring them (as traditional mergesort or heapsort would), Timsort rides them to faster results.

The Minimum Run

You want runs big enough to keep merges efficient. Tiny runs = too many merges = overhead.

So Timsort enforces a **minimum run length** (usually ~ 32 , depends on n). If it finds a run shorter than this, it extends it (using binary insertion sort).

This guarantees fewer, larger runs.

The Stack of Runs

Once Timsort has identified runs, it keeps them on a stack. But it doesn't just merge arbitrarily.

It enforces **invariants** on the sizes of runs on the stack to keep merges balanced.

Specifically, let's call the top runs on the stack A, B, and C (where A is the top):

- Invariant 1: $\text{len}(C) > \text{len}(B) + \text{len}(A)$
- Invariant 2: $\text{len}(B) > \text{len}(A)$

If these are violated after pushing a new run, Timsort merges runs until the invariant holds.

Why?

To avoid pathological merges (e.g. repeatedly merging a huge run with tiny runs, which would be slow).

These rules keep the merge tree balanced $\rightarrow O(n \log n)$ worst case.

Merging Runs

Merging runs is like mergesort's merge step, but with a twist: **galloping**.

Galloping (a.k.a. exponential search):

When merging two runs, if one run wins several elements in a row (e.g. left run's elements keep being smaller), Timsort "gallops" ahead:

- Instead of linear one-by-one comparison, it jumps ahead exponentially to find the cut point.

This reduces comparisons in long stretches of domination.

Galloping adapts: if the data "wants" to gallop (long stretches from one run), it does so aggressively. If not, it falls back to normal merging.

This tuning is part of why Timsort is so fast on real-world data.

Stability

Timsort is **stable**:

- Equal items keep their relative order
- During merges, when elements are equal, Timsort chooses the first one

Stability is crucial for many applications (e.g. multi-key sorts).

Worst-Case Guarantees

Despite all the adaptive tricks, Timsort has provable $O(n \log n)$ worst-case behavior.

How?

- The stack invariants ensure the merge tree has logarithmic height
- Each merge is linear time
- Number of merges is $O(\log n)$

So total work is $O(n \log n)$.

Best-Case Behavior

Best case: already sorted list.

- Only one run detected
- No merges
- Just a single pass to verify order

$O(n)$ time.

Similarly, nearly sorted data yields very few runs → minimal merging.

Insertion Sort for Short Runs

Short runs are filled out using **binary insertion sort**.

Why?

- Insertion sort is great for small arrays
- Binary search reduces comparisons
- Ensures small runs are fully sorted before merging

This helps constant factors.

Real-World Performance

Timsort outperforms naive mergesort or quicksort on many real-world workloads because:

- It adapts to existing order
- It reduces unnecessary comparisons via galloping
- It maintains stability
- Its constant factors are excellent

It was so successful it was adopted by:

- Python (`list.sort`)

- Java (Arrays.sort for objects)
- Android

A Tiny Example

Consider sorting:

[2, 3, 4, 1, 0, 5, 6]

- Scan → runs:
 - [2, 3, 4]
 - [1, 0] (reverse to [0, 1])
 - [5, 6]
- Merge rules:
 - Invariant check
 - Merge smallest runs first if needed

Final sorted list after merging the runs.

Why Not Quicksort?

Quicksort is not stable. Also:

- Worst-case time $O(n^2)$ unless carefully engineered
- Tends to ignore existing order

Timsort was specifically designed for:

- Stability
- Exploiting partial order
- Guaranteed worst-case

References

- Python source: listobject.c
- Original mailing list post: [Python-Dev Archive 2002]
- "TimSort: A very clever stable sorting algorithm" – various online writeups
- Java's adaptation notes

with, match, and else Blocks

Context managers may end up being almost as important as the subroutine itself. We’ve only scratched the surface with them. [...] Basic has a `with` statement, there are `with` statements in lots of languages. But they don’t do the same thing, they all do something very shallow, they save you from repeated dotted [attribute] lookups, they don’t do setup and tear down. Just because it’s the same name don’t think it’s the same thing. The `with` statement is a very big deal.¹

—Raymond Hettinger, eloquent Python evangelist

This chapter is about control flow features that are not so common in other languages, and for this reason tend to be overlooked or underused in Python. They are:

- The `with` statement and context manager protocol
- Pattern matching with `match/case`
- The `else` clause in `for`, `while`, and `try` statements

The `with` statement sets up a temporary context and reliably tears it down, under the control of a context manager object. This prevents errors and reduces boilerplate code, making APIs at the same time safer and easier to use. Python programmers are finding lots of uses for `with` blocks beyond automatic file closing.

We’ve seen pattern matching in previous chapters, but here we’ll see how the grammar of a language can be expressed as sequence patterns. That observation explains why `match/case` is an effective tool to create language processors that are easy to understand and extend. We’ll study a complete interpreter for a small but functional

¹ PyCon US 2013 keynote: “[What Makes Python Awesome](#)”; the part about `with` starts at 23:00 and ends at 26:15.

subset of the Scheme language. The same ideas could be applied to develop a template language or a DSL (Domain-Specific Language) to encode business rules in a larger system.

The `else` clause is not a big deal, but it does help convey intention when properly used together with `for`, `while`, and `try`.

What's New in This Chapter

“Pattern Matching in `lis.py`: A Case Study” on page 669 is a new section.

I updated “The `contextlib` Utilities” on page 663 to cover a few features of the `contextlib` module added since Python 3.6, and the new parenthesized context managers syntax introduced in Python 3.10.

Let's start with the powerful `with` statement.

Context Managers and `with` Blocks

Context manager objects exist to control a `with` statement, just like iterators exist to control a `for` statement.

The `with` statement was designed to simplify some common uses of `try/finally`, which guarantees that some operation is performed after a block of code, even if the block is terminated by `return`, an exception, or a `sys.exit()` call. The code in the `finally` clause usually releases a critical resource or restores some previous state that was temporarily changed.

The Python community is finding new, creative uses for context managers. Some examples from the standard library are:

- Managing transactions in the `sqlite3` module—see “Using the connection as a context manager”.
- Safely handling locks, conditions, and semaphores—as described in the `threading` module documentation.
- Setting up custom environments for arithmetic operations with `Decimal` objects—see the `decimal.localcontext` documentation.
- Patching objects for testing—see the `unittest.mock.patch` function.

The context manager interface consists of the `__enter__` and `__exit__` methods. At the top of the `with`, Python calls the `__enter__` method of the context manager object. When the `with` block completes or terminates for any reason, Python calls `__exit__` on the context manager object.

The most common example is making sure a file object is closed. [Example 18-1](#) is a detailed demonstration of using `with` to close a file.

Example 18-1. Demonstration of a file object as a context manager

```
>>> with open('mirror.py') as fp: ❶
...     src = fp.read(60) ❷
...
>>> len(src)
60
>>> fp ❸
<_io.TextIOWrapper name='mirror.py' mode='r' encoding='UTF-8'>
>>> fp.closed, fp.encoding ❹
(True, 'UTF-8')
>>> fp.read(60) ❺
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: I/O operation on closed file.
```

- ❶ `fp` is bound to the opened text file because the file's `__enter__` method returns `self`.
- ❷ Read 60 Unicode characters from `fp`.
- ❸ The `fp` variable is still available—`with` blocks don't define a new scope, as functions do.
- ❹ We can read the attributes of the `fp` object.
- ❺ But we can't read more text from `fp` because at the end of the `with` block, the `TextIOWrapper.__exit__` method was called, and it closed the file.

The first callout in [Example 18-1](#) makes a subtle but crucial point: the context manager object is the result of evaluating the expression after `with`, but the value bound to the target variable (in the `as` clause) is the result returned by the `__enter__` method of the context manager object.

It just happens that the `open()` function returns an instance of `TextIOWrapper`, and its `__enter__` method returns `self`. But in a different class, the `__enter__` method may also return some other object instead of the context manager instance.

When control flow exits the `with` block in any way, the `__exit__` method is invoked on the context manager object, not on whatever was returned by `__enter__`.

The `as` clause of the `with` statement is optional. In the case of `open`, we always need it to get a reference to the file, so that we can call methods on it. But some context managers return `None` because they have no useful object to give back to the user.

Example 18-2 shows the operation of a perfectly frivolous context manager designed to highlight the distinction between the context manager and the object returned by its `__enter__` method.

Example 18-2. Test-driving the LookingGlass context manager class

```
>>> from mirror import LookingGlass
>>> with LookingGlass() as what: ❶
...     print('Alice, Kitty and Snowdrop') ❷
...     print(what)
...
pordwonS dna yttik ,ecilA
YKCOWREBBAJ
>>> what ❸
'JABBERWOCKY'
>>> print('Back to normal.') ❹
Back to normal.
```

- ❶ The context manager is an instance of `LookingGlass`; Python calls `__enter__` on the context manager and the result is bound to `what`.
- ❷ Print a `str`, then the value of the target variable `what`. The output of each `print` will come out reversed.
- ❸ Now the `with` block is over. We can see that the value returned by `__enter__`, held in `what`, is the string `'JABBERWOCKY'`.
- ❹ Program output is no longer reversed.

Example 18-3 shows the implementation of `LookingGlass`.

Example 18-3. mirror.py: code for the LookingGlass context manager class

```
import sys

class LookingGlass:

    def __enter__(self): ❶
        self.original_write = sys.stdout.write ❷
        sys.stdout.write = self.reverse_write ❸
        return 'JABBERWOCKY' ❹

    def reverse_write(self, text): ❺
```

```

self.original_write(text[::-1])

def __exit__(self, exc_type, exc_value, traceback): ❹
    sys.stdout.write = self.original_write ❺
    if exc_type is ZeroDivisionError: ❸
        print('Please DO NOT divide by zero!')
    return True ❹
❺

```

- ❶ Python invokes `__enter__` with no arguments besides `self`.
- ❷ Hold the original `sys.stdout.write` method, so we can restore it later.
- ❸ Monkey-patch `sys.stdout.write`, replacing it with our own method.
- ❹ Return the 'JABBERWOCKY' string just so we have something to put in the target variable `what`.
- ❺ Our replacement to `sys.stdout.write` reverses the `text` argument and calls the original implementation.
- ❻ Python calls `__exit__` with `None`, `None`, `None` if all went well; if an exception is raised, the three arguments get the exception data, as described after this example.
- ❼ Restore the original method to `sys.stdout.write`.
- ❽ If the exception is not `None` and its type is `ZeroDivisionError`, print a message...
- ❾ ...and return `True` to tell the interpreter that the exception was handled.
- ❿ If `__exit__` returns `None` or any *falsy* value, any exception raised in the `with` block will be propagated.



When real applications take over standard output, they often want to replace `sys.stdout` with another file-like object for a while, then switch back to the original. The `contextlib.redirect_stdout` context manager does exactly that: just pass it the file-like object that will stand in for `sys.stdout`.

The interpreter calls the `__enter__` method with no arguments—beyond the implicit `self`. The three arguments passed to `__exit__` are:

`exc_type`

The exception class (e.g., `ZeroDivisionError`).

`exc_value`

The exception instance. Sometimes, parameters passed to the exception constructor—such as the error message—can be found in `exc_value.args`.

`traceback`

A traceback object.²

For a detailed look at how a context manager works, see [Example 18-4](#), where `LookingGlass` is used outside of a `with` block, so we can manually call its `__enter__` and `__exit__` methods.

Example 18-4. Exercising `LookingGlass` without a `with` block

```
>>> from mirror import LookingGlass
>>> manager = LookingGlass() ❶
>>> manager # doctest: +ELLIPSIS
<mirror.LookingGlass object at 0x...>
>>> monster = manager.__enter__() ❷
>>> monster == 'JABBERWOCKY' ❸
eurT
>>> monster
'YKCOWREBBAJ'
>>> manager # doctest: +ELLIPSIS
>... ta tcejbo ssalGgnikool.rorrim<
>>> manager.__exit__(None, None, None) ❹
>>> monster
'JABBERWOCKY'
```

- ❶ Instantiate and inspect the `manager` instance.
- ❷ Call the manager's `__enter__` method and store result in `monster`.
- ❸ `monster` is the string `'JABBERWOCKY'`. The True identifier appears reversed because all output via `stdout` goes through the `write` method we patched in `__enter__`.
- ❹ Call `manager.__exit__` to restore the previous `stdout.write`.

² The three arguments received by `self` are exactly what you get if you call `sys.exc_info()` in the `finally` block of a `try/finally` statement. This makes sense, considering that the `with` statement is meant to replace most uses of `try/finally`, and calling `sys.exc_info()` was often necessary to determine what clean-up action would be required.



Parenthesized Context Managers in Python 3.10

Python 3.10 adopted a new, more powerful parser, allowing new syntax beyond what was possible with the older LL(1) parser. One syntax enhancement was to allow parenthesized context managers, like this:

```
with (  
    CtxManager1() as example1,  
    CtxManager2() as example2,  
    CtxManager3() as example3,  
):  
    ...
```

Prior to 3.10, we'd have to write that as nested with blocks.

The standard library includes the `contextlib` package with handy functions, classes, and decorators for building, combining, and using context managers.

The contextlib Utilities

Before rolling your own context manager classes, take a look at `contextlib`—“Utilities for with-statement contexts” in the Python documentation. Maybe what you are about to build already exists, or there is a class or some callable that will make your job easier.

Besides the `redirect_stdout` context manager mentioned right after [Example 18-3](#), `redirect_stderr` was added in Python 3.5—it does the same as the former, but for output directed to `stderr`.

The `contextlib` package also includes:

`closing`

A function to build context managers out of objects that provide a `close()` method but don't implement the `__enter__`/`__exit__` interface.

`suppress`

A context manager to temporarily ignore exceptions given as arguments.

`nullcontext`

A context manager that does nothing, to simplify conditional logic around objects that may not implement a suitable context manager. It serves as a stand-in when conditional code before the with block may or may not provide a context manager for the with statement—added in Python 3.7.

The `contextlib` module provides classes and a decorator that are more widely applicable than the decorators just mentioned:

`@contextmanager`

A decorator that lets you build a context manager from a simple generator function, instead of creating a class and implementing the interface. See “Using `@contextmanager`” on page 664.

`AbstractContextManager`

An ABC that formalizes the context manager interface, and makes it a bit easier to create context manager classes by subclassing—added in Python 3.6.

`ContextDecorator`

A base class for defining class-based context managers that can also be used as function decorators, running the entire function within a managed context.

`ExitStack`

A context manager that lets you enter a variable number of context managers. When the `with` block ends, `ExitStack` calls the stacked context managers’ `__exit__` methods in LIFO order (last entered, first exited). Use this class when you don’t know beforehand how many context managers you need to enter in your `with` block; for example, when opening all files from an arbitrary list of files at the same time.

With Python 3.7, `contextlib` added `AbstractAsyncContextManager`, `@asyncontextmanager`, and `AsyncExitStack`. They are similar to the equivalent utilities without the `async` part of the name, but designed for use with the new `async with` statement, covered in [Chapter 21](#).

The most widely used of these utilities is the `@contextmanager` decorator, so it deserves more attention. That decorator is also interesting because it shows a use for the `yield` statement unrelated to iteration.

Using `@contextmanager`

The `@contextmanager` decorator is an elegant and practical tool that brings together three distinctive Python features: a function decorator, a generator, and the `with` statement.

Using `@contextmanager` reduces the boilerplate of creating a context manager: instead of writing a whole class with `__enter__`/`__exit__` methods, you just implement a generator with a single `yield` that should produce whatever you want the `__enter__` method to return.

In a generator decorated with `@contextmanager`, `yield` splits the body of the function in two parts: everything before the `yield` will be executed at the beginning of the `with` block when the interpreter calls `__enter__`; the code after `yield` will run when `__exit__` is called at the end of the block.

Example 18-5 replaces the LookingGlass class from **Example 18-3** with a generator function.

Example 18-5. mirror_gen.py: a context manager implemented with a generator

```
import contextlib
import sys

@contextlib.contextmanager ❶
def looking_glass():
    original_write = sys.stdout.write ❷

    def reverse_write(text): ❸
        original_write(text[::-1])

    sys.stdout.write = reverse_write ❹
    yield 'JABBERWOCKY' ❺
    sys.stdout.write = original_write ❻
```

- ❶ Apply the contextmanager decorator.
- ❷ Preserve the original sys.stdout.write method.
- ❸ reverse_write can call original_write later because it is available in its closure.
- ❹ Replace sys.stdout.write with reverse_write.
- ❺ Yield the value that will be bound to the target variable in the as clause of the with statement. The generator pauses at this point while the body of the with executes.
- ❻ When control exits the with block, execution continues after the yield; here the original sys.stdout.write is restored.

Example 18-6 shows the looking_glass function in operation.

Example 18-6. Test-driving the looking_glass context manager function

```
>>> from mirror_gen import looking_glass
>>> with looking_glass() as what: ❶
...     print('Alice, Kitty and Snowdrop')
...     print(what)
...
pordwonS dna yttiK ,ecila
YKCOWREBBAJ
>>> what
'JABBERWOCKY'
```

```
>>> print('back to normal')
back to normal
```

- ❶ The only difference from [Example 18-2](#) is the name of the context manager: `looking_glass` instead of `LookingGlass`.

The `contextlib.contextmanager` decorator wraps the function in a class that implements the `__enter__` and `__exit__` methods.³

The `__enter__` method of that class:

1. Calls the generator function to get a generator object—let’s call it `gen`.
2. Calls `next(gen)` to drive it to the `yield` keyword.
3. Returns the value yielded by `next(gen)`, to allow the user to bind it to a variable in the `with/as` form.

When the `with` block terminates, the `__exit__` method:

1. Checks whether an exception was passed as `exc_type`; if so, `gen.throw(exception)` is invoked, causing the exception to be raised in the `yield` line inside the generator function body.
2. Otherwise, `next(gen)` is called, resuming the execution of the generator function body after the `yield`.

[Example 18-5](#) has a flaw: if an exception is raised in the body of the `with` block, the Python interpreter will catch it and raise it again in the `yield` expression inside `looking_glass`. But there is no error handling there, so the `looking_glass` generator will terminate without ever restoring the original `sys.stdout.write` method, leaving the system in an invalid state.

[Example 18-7](#) adds special handling of the `ZeroDivisionError` exception, making it functionally equivalent to the class-based [Example 18-3](#).

Example 18-7. `mirror_gen_exc.py`: generator-based context manager implementing exception handling—same external behavior as [Example 18-3](#)

```
import contextlib
import sys

@contextlib.contextmanager
```

³ The actual class is named `_GeneratorContextManager`. If you want to see exactly how it works, read its [source code](#) in `Lib/contextlib.py` in Python 3.10.

```
def looking_glass():
    original_write = sys.stdout.write

    def reverse_write(text):
        original_write(text[::-1])

    sys.stdout.write = reverse_write
    msg = '' ❶
    try:
        yield 'JABBERWOCKY'
    except ZeroDivisionError: ❷
        msg = 'Please DO NOT divide by zero!'
    finally:
        sys.stdout.write = original_write ❸
        if msg:
            print(msg) ❹
```

- ❶ Create a variable for a possible error message; this is the first change in relation to [Example 18-5](#).
- ❷ Handle `ZeroDivisionError` by setting an error message.
- ❸ Undo monkey-patching of `sys.stdout.write`.
- ❹ Display error message, if it was set.

Recall that the `__exit__` method tells the interpreter that it has handled the exception by returning a truthy value; in that case, the interpreter suppresses the exception. On the other hand, if `__exit__` does not explicitly return a value, the interpreter gets the usual `None`, and propagates the exception. With `@contextmanager`, the default behavior is inverted: the `__exit__` method provided by the decorator assumes any exception sent into the generator is handled and should be suppressed.



Having a `try/finally` (or a `with` block) around the `yield` is an unavoidable price of using `@contextmanager`, because you never know what the users of your context manager are going to do inside the `with` block.⁴

⁴ This tip is quoted literally from a comment by Leonardo Rochael, one of the tech reviewers for this book. Nicely said, Leo!

A little-known feature of `@contextmanager` is that the generators decorated with it can also be used as decorators themselves.⁵ That happens because `@contextmanager` is implemented with the `contextlib.ContextDecorator` class.

Example 18-8 shows the `looking_glass` context manager from **Example 18-5** used as decorator.

Example 18-8. The `looking_glass` context manager also works as a decorator

```
>>> @looking_glass()
... def verse():
...     print('The time has come')
...
>>> verse() ❶
emoc sah emit ehT
>>> print('back to normal') ❷
back to normal
```

❶ `looking_glass` does its job before and after the body of `verse` runs.

❷ This confirms that the original `sys.write` was restored.

Contrast **Example 18-8** with **Example 18-6**, where `looking_glass` is used as a context manager.

An interesting real-life example of `@contextmanager` outside of the standard library is Martijn Pieters’ **in-place file rewriting using a context manager**. **Example 18-9** shows how it’s used.

Example 18-9. A context manager for rewriting files in place

```
import csv

with inplace(csvfilename, 'r', newline='') as (infh, outfh):
    reader = csv.reader(infh)
    writer = csv.writer(outfh)

    for row in reader:
        row += ['new', 'columns']
        writer.writerow(row)
```

The `inplace` function is a context manager that gives you two handles—`infh` and `outfh` in the example—to the same file, allowing your code to read and write to it at

⁵ At least I and the other technical reviewers didn’t know it until Caleb Hattingh told us. Thanks, Caleb!

the same time. It's easier to use than the standard library's `fileinput.input` function (which also provides a context manager, by the way).

If you want to study Martijn's `inplace` source code (listed in [the post](#)), find the `yield` keyword: everything before it deals with setting up the context, which entails creating a backup file, then opening and yielding references to the readable and writable file handles that will be returned by the `__enter__` call. The `__exit__` processing after the `yield` closes the file handles and restores the file from the backup if something went wrong.

This concludes our overview of the `with` statement and context managers. Let's turn to `match/case` in the context of a complete example.

Pattern Matching in `lis.py`: A Case Study

In [“Pattern Matching Sequences in an Interpreter” on page 43](#) we saw examples of sequence patterns extracted from the `evaluate` function of Peter Norvig's `lis.py` interpreter, ported to Python 3.10. In this section I want to give a broader overview of how `lis.py` works, and also explore all the case clauses of `evaluate`, explaining not only the patterns but also what the interpreter does in each case.

Besides showing more pattern matching, I wrote this section for three reasons:

1. Norvig's `lis.py` is a beautiful example of idiomatic Python code.
2. The simplicity of Scheme is a master class of language design.
3. Learning how an interpreter works gave me a deeper understanding of Python and programming languages in general—interpreted or compiled.

Before looking at the Python code, let's get a little taste of Scheme so you can make sense of this case study—in case you haven't seen Scheme or Lisp before.

Scheme Syntax

In Scheme there is no distinction between expressions and statements, like we have in Python. Also, there are no infix operators. All expressions use prefix notation like `(+ x 13)` instead of `x + 13`. The same prefix notation is used for function calls—e.g., `(gcd x 13)`—and special forms—e.g., `(define x 13)`, which we'd write as the assignment statement `x = 13` in Python. The notation used by Scheme and most Lisp dialects is known as *S-expression*.⁶

⁶ People complain about too many parentheses in Lisp, but thoughtful indentation and a good editor mostly take care of that issue. The main readability problem is using the same `(f ...)` notation for function calls and special forms like `(define ...)`, `(if ...)`, and `(quote ...)` that don't behave at all like function calls.

Example 18-10 shows a simple example in Scheme.

Example 18-10. Greatest common divisor in Scheme

```
(define (mod m n)
  (- m (* n (quotient m n))))

(define (gcd m n)
  (if (= n 0)
      m
      (gcd n (mod m n))))

(display (gcd 18 45))
```

Example 18-10 shows three Scheme expressions: two function definitions—`mod` and `gcd`—and a call to `display`, which will output 9, the result of `(gcd 18 45)`. Example 18-11 is the same code in Python (shorter than an English explanation of the recursive *Euclidean algorithm*).

Example 18-11. Same as Example 18-10, written in Python

```
def mod(m, n):
    return m - (m // n * n)

def gcd(m, n):
    if n == 0:
        return m
    else:
        return gcd(n, mod(m, n))

print(gcd(18, 45))
```

In idiomatic Python, I’d use the `%` operator instead of reinventing `mod`, and it would be more efficient to use a `while` loop instead of recursion. But I wanted to show two function definitions, and make the examples as similar as possible, to help you read the Scheme code.

Scheme has no iterative control flow commands like `while` or `for`. Iteration is done with recursion. Note how there are no assignments in the Scheme and Python examples. Extensive use of recursion and minimal use of assignment are hallmarks of programming in a functional style.⁷

⁷ To make iteration through recursion practical and efficient, Scheme and other functional languages implement *proper tail calls*. For more about this, see “[Soapbox](#)” on page 691.

Now let's review the code of the Python 3.10 version of *lis.py*. The complete source code with tests is in the [18-with-match/lispy/py3.10/](#) directory of the GitHub repository [fluentpython/example-code-2e](#).

Imports and Types

Example 18-12 shows the first lines of *lis.py*. The use of `TypeAlias` and the `|` type union operator require Python 3.10.

Example 18-12. lis.py: top of the file

```
import math
import operator as op
from collections import ChainMap
from itertools import chain
from typing import Any, TypeAlias, NoReturn
```

```
Symbol: TypeAlias = str
Atom: TypeAlias = float | int | Symbol
Expression: TypeAlias = Atom | list
```

The types defined are:

Symbol

Just an alias for `str`. In *lis.py*, `Symbol` is used for identifiers; there is no string data type with operations such as slicing, splitting, etc.⁸

Atom

A simple syntactic element, such as a number or a `Symbol`—as opposed to a composite structure made of distinct parts, like a list.

Expression

The building blocks of Scheme programs are expressions made of atoms and lists, possibly nested.

The Parser

Norvig's parser is 36 lines of code showcasing the power of Python applied to handling the simple recursive syntax of S-expression—without string data, comments, macros, and other features of standard Scheme that make parsing more complicated (**Example 18-13**).

⁸ But Norvig's second interpreter, *lispy.py*, supports strings as a data type, as well as advanced features like syntactic macros, continuations, and proper tail calls. However, *lispy.py* is almost three times longer than *lis.py*—and much harder to understand.

Example 18-13. *lis.py*: the main parsing functions

```
def parse(program: str) -> Expression:
    "Read a Scheme expression from a string."
    return read_from_tokens(tokenize(program))

def tokenize(s: str) -> list[str]:
    "Convert a string into a list of tokens."
    return s.replace('(', ' ( ').replace(')', ' ) ').split()

def read_from_tokens(tokens: list[str]) -> Expression:
    "Read an expression from a sequence of tokens."
    # more parsing code omitted in book listing
```

The main function of that group is `parse`, which takes an S-expression as a `str` and returns an `Expression` object, as defined in [Example 18-12](#): an `Atom` or a list that may contain more atoms and nested lists.

Norvig uses a smart trick in `tokenize`: he adds spaces before and after each parenthesis in the input and then splits it, resulting in a list of syntactic tokens with `'('` and `')'` as separate tokens. This shortcut works because there is no string type in the little Scheme of *lis.py*, so every `'('` or `)'` is an expression delimiter. The recursive parsing code is in `read_from_tokens`, a 14-line function that you can read in the *fluentpython/example-code-2e* repository. I will skip it because I want to focus on the other parts of the interpreter.

Here are some doctests extracted from *lispy/py3.10/examples_test.py*:

```
>>> from lis import parse
>>> parse('1.5')
1.5
>>> parse('ni!')
'ni!'
>>> parse('(gcd 18 45)')
['gcd', 18, 45]
>>> parse('''
... (define double
...     (lambda (n)
...         (* n 2)))
... ''')
['define', 'double', ['lambda', ['n'], ['*', 'n', 2]]]
```

The parsing rules for this subset of Scheme are simple:

1. A token that looks like a number is parsed as a `float` or `int`.
2. Anything else that is not `'('` or `)'` is parsed as a `Symbol`—a `str` to be used as an identifier. This includes source text like `+`, `set!`, and `make-counter` that are valid identifiers in Scheme but not in Python.

- Expressions inside '(' and ')' are recursively parsed as lists containing atoms or as nested lists that may contain atoms and more nested lists.

Using the terminology of the Python interpreter, the output of `parse` is an AST (Abstract Syntax Tree): a convenient representation of the Scheme program as nested lists forming a tree-like structure, where the outermost list is the trunk, inner lists are the branches, and atoms are the leaves (Figure 18-1).

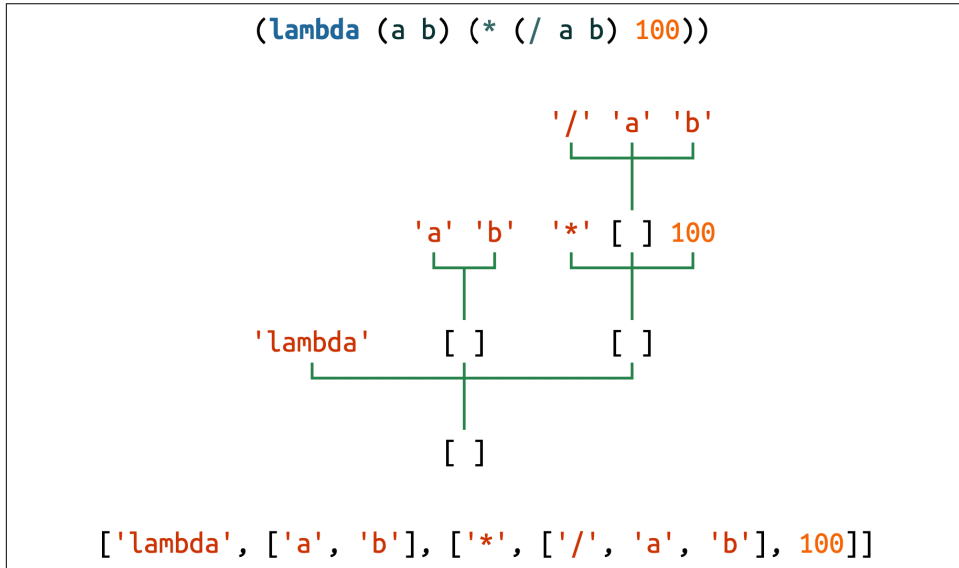


Figure 18-1. A Scheme `lambda` expression represented as source code (concrete syntax), as a tree, and as a sequence of Python objects (abstract syntax).

The Environment

The `Environment` class extends `collections.ChainMap`, adding a `change` method to update a value inside one of the chained dicts, which `ChainMap` instances hold in a list of mappings: the `self.maps` attribute. The `change` method is needed to support the Scheme (`set! ...`) form, described later; see Example 18-14.

Example 18-14. lis.py: the `Environment` class

```
class Environment(ChainMap[Symbol, Any]):
    "A ChainMap that allows changing an item in-place."

    def change(self, key: Symbol, value: Any) -> None:
        "Find where key is defined and change the value there."
        for map in self.maps:
            if key in map:
```

```

        map[key] = value # type: ignore[index]
    return
    raise KeyError(key)

```

Note that the `change` method only updates existing keys.⁹ Trying to change a key that is not found raises `KeyError`.

This doctest shows how `Environment` works:

```

>>> from lis import Environment
>>> inner_env = {'a': 2}
>>> outer_env = {'a': 0, 'b': 1}
>>> env = Environment(inner_env, outer_env)
>>> env['a'] ❶
2
>>> env['a'] = 111 ❷
>>> env['c'] = 222
>>> env
Environment({'a': 111, 'c': 222}, {'a': 0, 'b': 1})
>>> env.change('b', 333) ❸
>>> env
Environment({'a': 111, 'c': 222}, {'a': 0, 'b': 333})

```

- ❶ When reading values, `Environment` works as `ChainMap`: keys are searched in the nested mappings from left to right. That's why the value of `a` in the `outer_env` is shadowed by the value in `inner_env`.
- ❷ Assigning with `[]` overwrites or inserts new items, but always in the first mapping, `inner_env` in this example.
- ❸ `env.change('b', 333)` seeks the `'b'` key and assigns a new value to it in-place, in the `outer_env`.

Next is the `standard_env()` function, which builds and returns an `Environment` loaded with predefined functions, similar to Python's `__builtins__` module that is always available (Example 18-15).

Example 18-15. lis.py: `standard_env()` builds and returns the global environment

```

def standard_env() -> Environment:
    "An environment with some Scheme standard procedures."
    env = Environment()
    env.update(vars(math)) # sin, cos, sqrt, pi, ...

```

⁹ The `# type: ignore[index]` comment is there because of *typeshed* issue #6042, which is unresolved as I review this chapter. `ChainMap` is annotated as `MutableMapping`, but the type hint in the `maps` attribute says it's a list of `Mapping`, indirectly making the whole `ChainMap` immutable as far as `Mypy` is concerned.

```

env.update({
    '+': op.add,
    '-': op.sub,
    '*': op.mul,
    '/': op.truediv,
    # omitted here: more operator definitions
    'abs': abs,
    'append': lambda *args: list(chain(*args)),
    'apply': lambda proc, args: proc(*args),
    'begin': lambda *x: x[-1],
    'car': lambda x: x[0],
    'cdr': lambda x: x[1:],
    # omitted here: more function definitions
    'number?': lambda x: isinstance(x, (int, float)),
    'procedure?': callable,
    'round': round,
    'symbol?': lambda x: isinstance(x, Symbol),
})
return env

```

To summarize, the env mapping is loaded with:

- All functions from Python’s `math` module
- Selected operators from Python’s `op` module
- Simple but powerful functions built with Python’s `lambda`
- Python built-ins renamed, like `callable` as `procedure?`, or directly mapped, like `round`

The REPL

Norvig’s REPL (read-eval-print-loop) is easy to understand but not user-friendly (see [Example 18-16](#)). If no command-line arguments are given to `lis.py`, the `repl()` function is invoked by `main()`—defined at the end of the module. At the `lis.py>` prompt, we must enter correct and complete expressions; if we forget to close one parenthesis, `lis.py` crashes.¹⁰

¹⁰ As I studied Norvig’s `lis.py` and `lispy.py`, I started a fork named *mylis* that adds some features, including a REPL that accepts partial S-expressions and prompts for the continuation, similar to how Python’s REPL knows we are not finished and presents the secondary prompt (`...`) until we enter a complete expression or statement that can be evaluated. *mylis* also handles a few errors gracefully, but it’s still easy to crash. It’s not nearly as robust as Python’s REPL.

Example 18-16. The REPL functions

```
def repl(prompt: str = 'lis.py> ') -> NoReturn:
    "A prompt-read-eval-print loop."
    global_env = Environment({}, standard_env())
    while True:
        ast = parse(input(prompt))
        val = evaluate(ast, global_env)
        if val is not None:
            print(lispstr(val))

def lispstr(exp: object) -> str:
    "Convert a Python object back into a Lisp-readable string."
    if isinstance(exp, list):
        return '(' + ' '.join(map(lispstr, exp)) + ')'
    else:
        return str(exp)
```

Here is a quick explanation about these two functions:

`repl(prompt: str = 'lis.py> ') -> NoReturn`

Calls `standard_env()` to provide built-in functions for the global environment, then enters an infinite loop, reading and parsing each input line, evaluating it in the global environment, and displaying the result—unless it's `None`. The `global_env` may be modified by `evaluate`. For example, when a user defines a new global variable or named function, it is stored in the first mapping of the environment—the empty dict in the `Environment` constructor call in the first line of `repl`.

`lispstr(exp: object) -> str`

The inverse function of `parse`: given a Python object representing an expression, `parse` returns the Scheme source code for it. For example, given `['+', 2, 3]`, the result is `('+ 2 3')`.

The Evaluator

Now we can appreciate the beauty of Norvig's expression evaluator—made a little prettier with `match/case`. The `evaluate` function in [Example 18-17](#) takes an Expression built by `parse` and an `Environment`.

The body of `evaluate` is a single `match` statement with an expression `exp` as the subject. The case patterns express the syntax and semantics of Scheme with amazing clarity.

Example 18-17. evaluate takes an expression and computes its value

```
KEYWORDS = ['quote', 'if', 'lambda', 'define', 'set!']

def evaluate(exp: Expression, env: Environment) -> Any:
    "Evaluate an expression in an environment."
    match exp:
        case int(x) | float(x):
            return x
        case Symbol(var):
            return env[var]
        case ['quote', x]:
            return x
        case ['if', test, consequence, alternative]:
            if evaluate(test, env):
                return evaluate(consequence, env)
            else:
                return evaluate(alternative, env)
        case ['lambda', [*parms], *body] if body:
            return Procedure(parms, body, env)
        case ['define', Symbol(name), value_exp]:
            env[name] = evaluate(value_exp, env)
        case ['define', [Symbol(name), *parms], *body] if body:
            env[name] = Procedure(parms, body, env)
        case ['set!', Symbol(name), value_exp]:
            env.change(name, evaluate(value_exp, env))
        case [func_exp, *args] if func_exp not in KEYWORDS:
            proc = evaluate(func_exp, env)
            values = [evaluate(arg, env) for arg in args]
            return proc(*values)
        case _:
            raise SyntaxError(lispstr(exp))
```

Let's study each case clause and what it does. In some cases I added comments showing an S-expression that would match the pattern when parsed into a Python list. Doctests extracted from *examples_test.py* demonstrate each case.

Evaluating numbers

```
case int(x) | float(x):
    return x
```

Subject:

Instance of `int` or `float`.

Action:

Return value as is.

Example:

```
>>> from lis import parse, evaluate, standard_env
>>> evaluate(parse('1.5'), {})
1.5
```

Evaluating symbols

```
case Symbol(var):
    return env[var]
```

Subject:

Instance of `Symbol`, i.e., a str used as an identifier.

Action:

Look up `var` in `env` and return its value.

Examples:

```
>>> evaluate(parse('+'), standard_env())
<built-in function add>
>>> evaluate(parse('ni!'), standard_env())
Traceback (most recent call last):
...
KeyError: 'ni!'
```

(quote ...)

The `quote` special form treats atoms and lists as data instead of expressions to be evaluated.

```
# (quote (99 bottles of beer))
case ['quote', x]:
    return x
```

Subject:

List starting with the symbol `'quote'`, followed by one expression `x`.

Action:

Return `x` without evaluating it.

Examples:

```
>>> evaluate(parse('(quote no-such-name)'), standard_env())
'no-such-name'
>>> evaluate(parse('(quote (99 bottles of beer))'), standard_env())
[99, 'bottles', 'of', 'beer']
>>> evaluate(parse('(quote (/ 10 0))'), standard_env())
['/', 10, 0]
```

Without `quote`, each expression in the test would raise an error:

- `no-such-name` would be looked up in the environment, raising `KeyError`
- `(99 bottles of beer)` cannot be evaluated because the number 99 is not a `Symbol` naming a special form, operator, or function
- `(/ 10 0)` would raise `ZeroDivisionError`

Why Languages Have Reserved Keywords

Although simple, `quote` cannot be implemented as a function in Scheme. Its special power is to prevent the interpreter from evaluating `(f 10)` in the expression `(quote (f 10))`: the result is simply a list with a `Symbol` and an `int`. In contrast, in a function call like `(abs (f 10))`, the interpreter evaluates `(f 10)` before invoking `abs`. That's why `quote` is a reserved keyword: it must be handled as a special form.

In general, reserved keywords are needed:

- To introduce specialized evaluation rules, as in `quote` and `lambda`—which don't evaluate any of their subexpressions
- To change the control flow, as in `if` and function calls—which also have special evaluation rules
- To manage the environment, as in `define` and `set`

This is also why Python, and programming languages in general, need reserved keywords. Think about Python's `def`, `if`, `yield`, `import`, `del`, and what they do.

(if ...)

```
# (if (< x 0) 0 x)
case ['if', test, consequence, alternative]:
    if evaluate(test, env):
        return evaluate(consequence, env)
    else:
        return evaluate(alternative, env)
```

Subject:

List starting with 'if' followed by three expressions: test, consequence, and alternative.

Action:

Evaluate test:

- If true, evaluate consequence and return its value.
- Otherwise, evaluate alternative and return its value.

Examples:

```
>>> evaluate(parse('(if (= 3 3) 1 0)'), standard_env())
1
>>> evaluate(parse('(if (= 3 4) 1 0)'), standard_env())
0
```

The consequence and alternative branches must be single expressions. If more than one expression is needed in a branch, you can combine them with `(begin exp1 exp2...)`, provided as a function in *lis.py*—see [Example 18-15](#).

(lambda ...)

Scheme's `lambda` form defines anonymous functions. It doesn't suffer from the limitations of Python's `lambda`: any function that can be written in Scheme can be written using the `(lambda ...)` syntax.

```
# (lambda (a b) (/ (+ a b) 2))
case ['lambda' [*parms], *body] if body:
    return Procedure(parms, body, env)
```

Subject:

List starting with 'lambda', followed by:

- List of zero or more parameter names.
- One or more expressions collected in body (the guard ensures that body is not empty).

Action:

Create and return a new `Procedure` instance with the parameter names, the list of expressions as the body, and the current environment.

Example:

```
>>> expr = '(lambda (a b) (* (/ a b) 100))'
>>> f = evaluate(parse(expr), standard_env())
>>> f # doctest: +ELLIPSIS
<lis.Procedure object at 0x...>
>>> f(15, 20)
75.0
```

The `Procedure` class implements the concept of a closure: a callable object holding parameter names, a function body, and a reference to the environment in which the function is defined. We'll study the code for `Procedure` in a moment.

(define ...)

The `define` keyword is used in two different syntactic forms. The simplest is:

```
# (define half (/ 1 2))
case ['define', Symbol(name), value_exp]:
    env[name] = evaluate(value_exp, env)
```

Subject:

List starting with `'define'`, followed by a `Symbol` and an expression.

Action:

Evaluate the expression and put its value into `env`, using `name` as key.

Example:

```
>>> global_env = standard_env()
>>> evaluate(parse('(define answer (* 7 6))'), global_env)
>>> global_env['answer']
42
```

The doctest for this case creates a `global_env` so that we can verify that `evaluate` puts `answer` into that `Environment`.

We can use that simple `define` form to create variables or to bind names to anonymous functions, using `(lambda ...)` as the `value_exp`.

Standard Scheme provides a shortcut for defining named functions. That's the second `define` form:

```
# (define (average a b) (/ (+ a b) 2))
case ['define', [Symbol(name), *parms], *body] if body:
    env[name] = Procedure(parms, body, env)
```

Subject:

List starting with `'define'`, followed by:

- A list starting with a `Symbol(name)`, followed by zero or more items collected into a list named `parms`.
- One or more expressions collected in `body` (the guard ensures that `body` is not empty).

Action:

- Create a new `Procedure` instance with the parameter names, the list of expressions as the body, and the current environment.
- Put the `Procedure` into `env`, using `name` as key.

The doctest in [Example 18-18](#) defines a function named `%` that computes a percentage and adds it to the `global_env`.

Example 18-18. Defining a function named % that computes a percentage

```
>>> global_env = standard_env()
>>> percent = '(define (% a b) (* (/ a b) 100))'
>>> evaluate(parse(percent), global_env)
>>> global_env['%'] # doctest: +ELLIPSIS
<lis.Procedure object at 0x...>
>>> global_env['%'](170, 200)
85.0
```

After calling `evaluate`, we check that `%` is bound to a `Procedure` that takes two numeric arguments and returns a percentage.

The pattern for the second `define` case does not enforce that the items in `parms` are all `Symbol` instances. I'd have to check that before building the `Procedure`, but I didn't—to keep the code as easy to follow as Norvig's.

(set! ...)

The `set!` form changes the value of a previously defined variable.¹¹

```
# (set! n (+ n 1))
case ['set!', Symbol(name), value_exp]:
    env.change(name, evaluate(value_exp, env))
```

Subject:

List starting with `'set!'`, followed by a `Symbol` and an expression.

Action:

Update the value of `name` in `env` with the result of evaluating the expression.

The `Environment.change` method traverses the chained environments from local to global, and updates the first occurrence of `name` with the new value. If we were not implementing the `'set!'` keyword, we could use Python's `ChainMap` as the `Environment` type everywhere in this interpreter.

¹¹ Assignment is one of the first features taught in many programming tutorials, but `set!` only appears on page 220 of the best known Scheme book, *Structure and Interpretation of Computer Programs, 2nd ed.*, by Abelson et al. (MIT Press), a.k.a. SICP or the “Wizard Book.” Coding in a functional style can take us very far without the state changes that are typical of imperative and object-oriented programming.

Python's nonlocal and Scheme's set! Address the Same Issue

The use of the `set!` form is related to the use of the `nonlocal` keyword in Python: declaring `nonlocal x` allows `x = 10` to update a previously defined `x` variable outside of the local scope. Without a `nonlocal x` declaration, `x = 10` will always create a local variable in Python, as we saw in “The nonlocal Declaration” on page 315.

Similarly, `(set! x 10)` updates a previously defined `x` that may be outside of the local environment of the function. In contrast, the variable `x` in `(define x 10)` is always a local variable, created or updated in the local environment.

Both `nonlocal` and `(set! ...)` are needed to update program state held in variables within a closure. Example 9-13 demonstrated the use of `nonlocal` to implement a function to compute a running average, holding an item count and total in a closure. Here is that same idea, written in the Scheme subset of *lis.py*:

```
(define (make-averager)
  (define count 0)
  (define total 0)
  (lambda (new-value)
    (set! count (+ count 1))
    (set! total (+ total new-value))
    (/ total count)
  )
)
(define avg (make-averager)) ❶
(avg 10) ❷
(avg 11) ❸
(avg 15) ❹
```

- ❶ Creates a new closure with the inner function defined by `lambda`, and the variables `count` and `total` initialized to 0; binds the closure to `avg`.
- ❷ Returns 10.0.
- ❸ Returns 10.5.
- ❹ Returns 12.0.

The preceding code is one of the tests in *lispy/py3.10/examples_test.py*.

Now we get to a function call.

Function call

```
# (gcd (* 2 105) 84)
case [func_exp, *args] if func_exp not in KEYWORDS:
    proc = evaluate(func_exp, env)
    values = [evaluate(arg, env) for arg in args]
    return proc(*values)
```

Subject:

List with one or more items.

The guard ensures that `func_exp` is not one of `['quote', 'if', 'define', 'lambda', 'set!']`—listed right before `evaluate` in [Example 18-17](#).

The pattern matches any list with one or more expressions, binding the first expression to `func_exp` and the rest to `args` as a list, which may be empty.

Action:

- Evaluate `func_exp` to obtain a function `proc`.
- Evaluate each item in `args` to build a list of argument values.
- Call `proc` with the values as separate arguments, returning the result.

Example:

```
>>> evaluate(parse('(% (* 12 14) (- 500 100))'), global_env)
42.0
```

This doctest continues from [Example 18-18](#): it assumes `global_env` has a function named `%`. The arguments given to `%` are arithmetic expressions, to emphasize that the arguments are evaluated before the function is called.

The guard in this case is needed because `[func_exp, *args]` matches any sequence subject with one or more items. However, if `func_exp` is a keyword, and the subject did not match any previous case, then it is really a syntax error.

Catch syntax errors

If the subject `exp` does not match any of the previous cases, the catch-all case raises a `SyntaxError`:

```
case _:
    raise SyntaxError(lispstr(exp))
```

Here is an example of a malformed `(lambda ...)` reported as a `SyntaxError`:

```
>>> evaluate(parse('(lambda is not like this)'), standard_env())
Traceback (most recent call last):
...
SyntaxError: (lambda is not like this)
```

If the case for function call did not have that guard rejecting keywords, the `(lambda is not like this)` expression would be handled as a function call, which would raise `KeyError` because `'lambda'` is not part of the environment—just like `lambda` is not a Python built-in function.

Procedure: A Class Implementing a Closure

The `Procedure` class could very well be named `Closure`, because that's what it represents: a function definition together with an environment. The function definition includes the name of the parameters and the expressions that make up the body of the function. The environment is used when the function is called to provide the values of the *free variables*: variables that appear in the body of the function but are not parameters, local variables, or global variables. We saw the concepts of *closure* and *free variable* in “Closures” on page 311.

We learned how to use closures in Python, but now we can dive deeper and see how a closure is implemented in *lis.py*:

```
class Procedure:
    "A user-defined Scheme procedure."

    def __init__(1
        self, parms: list[Symbol], body: list[Expression], env: Environment
    ):
        self.parms = parms 2
        self.body = body
        self.env = env

    def __call__(self, *args: Expression) -> Any: 3
        local_env = dict(zip(self.parms, args)) 4
        env = Environment(local_env, self.env) 5
        for exp in self.body: 6
            result = evaluate(exp, env)
        return result 7
```

- ❶ Called when a function is defined by the `lambda` or `define` forms.
- ❷ Save the parameter names, body expressions, and environment for later use.
- ❸ Called by `proc(*values)` in the last line of the case `[func_exp, *args]` clause.
- ❹ Build `local_env` mapping `self.parms` as local variable names, and the given `args` as values.
- ❺ Build a new combined `env`, putting `local_env` first, and then `self.env`—the environment that was saved when the function was defined.

- ⑥ Iterate over each expression in `self.body`, evaluating it in the combined env.
- ⑦ Return the result of the last expression evaluated.

There are a couple of simple functions after `evaluate` in *lis.py*: `run` reads a complete Scheme program and executes it, and `main` calls `run` or `repl`, depending on the command line—similar to what Python does. I will not describe those functions because there’s nothing new in them. My goals were to share with you the beauty of Norvig’s little interpreter, to give more insight into how closures work, and to show how `match/case` is a great addition to Python.

To wrap up this extended section on pattern matching, let’s formalize the concept of an OR-pattern.

Using OR-patterns

A series of patterns separated by `|` is an *OR-pattern*: it succeeds if any of the subpatterns succeed. The pattern in “Evaluating numbers” on page 677 is an OR-pattern:

```
case int(x) | float(x):  
    return x
```

All subpatterns in an OR-pattern must use the same variables. This restriction is necessary to ensure that the variables are available to the guard expression and the case body, regardless of the subpattern that matched.



In the context of a `case` clause, the `|` operator has a special meaning. It does not trigger the `__or__` special method, which handles expressions like `a | b` in other contexts, where it is overloaded to perform operations such as set union or integer bitwise-or, depending on the operands.

An OR-pattern is not restricted to appear at the top level of a pattern. You can also use `|` in subpatterns. For example, if we wanted *lis.py* to accept the Greek letter λ (lambda)¹² as well as the `lambda` keyword, we can rewrite the pattern like this:

```
# (λ (a b) (/ (+ a b) 2) )  
case ['lambda' | 'λ', [*parms], *body] if body:  
    return Procedure(parms, body, env)
```

¹² The official Unicode name for λ (U+03BB) is GREEK SMALL LETTER LAMDA. This is not a typo: the character is named “lamda” without the “b” in the Unicode database. According to the English Wikipedia article “Lambda”, the Unicode Consortium adopted that spelling because of “preferences expressed by the Greek National Body.”

Now we can move to the third and last subject of this chapter: the unusual places where an `else` clause may appear in Python.

Do This, Then That: `else` Blocks Beyond `if`

This is no secret, but it is an underappreciated language feature: the `else` clause can be used not only in `if` statements but also in `for`, `while`, and `try` statements.

The semantics of `for/else`, `while/else`, and `try/else` are closely related, but very different from `if/else`. Initially, the word `else` actually hindered my understanding of these features, but eventually I got used to it.

Here are the rules:

`for`

The `else` block will run only if and when the `for` loop runs to completion (i.e., not if the `for` is aborted with a `break`).

`while`

The `else` block will run only if and when the `while` loop exits because the condition became *false* (i.e., not if the `while` is aborted with a `break`).

`try`

The `else` block will run only if no exception is raised in the `try` block. The [official docs](#) also state: “Exceptions in the `else` clause are not handled by the preceding `except` clauses.”

In all cases, the `else` clause is also skipped if an exception or a `return`, `break`, or `continue` statement causes control to jump out of the main block of the compound statement.



I think `else` is a very poor choice for the keyword in all cases except `if`. It implies an excluding alternative, like, “Run this loop, otherwise do that,” but the semantics for `else` in loops is the opposite: “Run this loop, then do that.” This suggests then as a better keyword—which would also make sense in the `try` context: “Try this, then do that.” However, adding a new keyword is a breaking change to the language—not an easy decision to make.

Using `else` with these statements often makes the code easier to read and saves the trouble of setting up control flags or coding extra `if` statements.

The use of `else` in loops generally follows the pattern of this snippet:

```
for item in my_list:
    if item.flavor == 'banana':
```

```

        break
    else:
        raise ValueError('No banana flavor found!')

```

In the case of try/except blocks, else may seem redundant at first. After all, the `after_call()` in the following snippet will run only if the `dangerous_call()` does not raise an exception, correct?

```

try:
    dangerous_call()
    after_call()
except OSError:
    log('OSError...')

```

However, doing so puts the `after_call()` inside the try block for no good reason. For clarity and correctness, the body of a try block should only have the statements that may generate the expected exceptions. This is better:

```

try:
    dangerous_call()
except OSError:
    log('OSError...')
else:
    after_call()

```

Now it's clear that the try block is guarding against possible errors in `dangerous_call()` and not in `after_call()`. It's also explicit that `after_call()` will only execute if no exceptions are raised in the try block.

In Python, try/except is commonly used for control flow, and not just for error handling. There's even an acronym/slogan for that documented in the [official Python glossary](#):

EAFP

Easier to ask for forgiveness than permission. This common Python coding style assumes the existence of valid keys or attributes and catches exceptions if the assumption proves false. This clean and fast style is characterized by the presence of many try and except statements. The technique contrasts with the *LBYL* style common to many other languages such as C.

The glossary then defines *LBYL*:

LBYL

Look before you leap. This coding style explicitly tests for pre-conditions before making calls or lookups. This style contrasts with the *EAFP* approach and is characterized by the presence of many if statements. In a multi-threaded environment, the *LBYL* approach can risk introducing a race condition between “the looking” and “the leaping.” For example, the code, if key in mapping: return mapping[key] can fail if another thread removes key from mapping after the test, but before the lookup. This issue can be solved with locks or by using the *EAFP* approach.

Given the EAFP style, it makes even more sense to know and use `else` blocks well in `try/except` statements.



When the `match` statement was discussed, some people (including me) thought it should also have an `else` clause. In the end it was decided that it wasn't needed because `case _:` does the same job.¹³

Now let's summarize the chapter.

Chapter Summary

This chapter started with context managers and the meaning of the `with` statement, quickly moving beyond its common use to automatically close opened files. We implemented a custom context manager: the `LookingGlass` class with the `__enter__`/`__exit__` methods, and saw how to handle exceptions in the `__exit__` method. A key point that Raymond Hettinger made in his PyCon US 2013 keynote is that `with` is not just for resource management; it's a tool for factoring out common setup and teardown code, or any pair of operations that need to be done before and after another procedure.¹⁴

We reviewed functions in the `contextlib` standard library module. One of them, the `@contextmanager` decorator, makes it possible to implement a context manager using a simple generator with one `yield`—a leaner solution than coding a class with at least two methods. We reimplemented the `LookingGlass` as a `looking_glass` generator function, and discussed how to do exception handling when using `@contextmanager`.

Then we studied Peter Norvig's elegant *lis.py*, a Scheme interpreter written in idiomatic Python, refactored to use `match/case` in `evaluate`—the function at the core of any interpreter. Understanding how `evaluate` works required reviewing a little bit of Scheme, a parser for S-expressions, a simple REPL, and the construction of nested scopes through an `Environment` subclass of `collection.ChainMap`. In the end, *lis.py* became a vehicle to explore much more than pattern matching. It shows how the different parts of an interpreter work together, illuminating core features of Python itself: why reserved keywords are necessary, how scoping rules work, and how closures are built and used.

¹³ Watching the discussion in the python-dev mailing list I thought one reason why `else` was rejected was the lack of consensus on how to indent it within `match`: should `else` be indented at the same level as `match`, or at the same level as `case`?

¹⁴ See [slide 21](#) in “Python is Awesome”.

Strategy Pattern Refactor of a Sorter

Problem & Motivation

Context

Suppose you have a Sorter class that sorts lists. Initially, it's hard-coded to a single algorithm (e.g., bubble sort). But you want flexibility to swap in other algorithms (quick sort, merge sort, etc.) without changing the Sorter class itself.

The Initial Problem

Original code (Java-like pseudocode):

```
class Sorter {  
    void sort(int[] data) {  
        // Bubble sort  
        for (int i = 0; i < data.length - 1; i++) {  
            for (int j = 0; j < data.length - i - 1; j++) {  
                if (data[j] > data[j+1]) {  
                    int temp = data[j];  
                    data[j] = data[j+1];  
                    data[j+1] = temp;  
                }  
            }  
        }  
    }  
}
```

Drawbacks:

- Algorithm is fixed.
- To add quicksort or mergesort, you must edit Sorter.
- Violates Open/Closed Principle (OCP): Software entities should be open for extension, but closed for modification.

The Goal

Decouple the sorting algorithm from the Sorter class so we can swap algorithms at runtime without modifying Sorter.

Enter the Strategy Pattern

The Strategy Pattern lets you:

- Define a family of algorithms.

- Encapsulate each one.
- Make them interchangeable.

Structure

- Strategy Interface: defines `sort()`.
- Concrete Strategies: implement `sort()` in different ways.
- Context (Sorter): uses a strategy to sort.